

IUPC / ICPC TEAM REFERENCE NOTEBOOK 2026

Jashore University of Science and Technology | Competitive Programming Reference Document

TABLE OF CONTENTS

1. Setup, Fast I/O & Debugging	2
2. Mathematics & Basic Algebra	2
3. Number Theory & Divisors	3
4. Combinatorics & Discrete Math	5
5. Data Structures	6
6. String Algorithms	9
7. Graph Theory	11
8. Dynamic Programming	14
9. Greedy, Searching & Techniques	16
10. Computational Geometry	16
11. Bit Hacks, Game Theory & STL Reference	17

ONSITE SAFETY NOTES

- Check indexing: most trees/graphs here are 1-indexed; most arrays/strings are 0-indexed.
- Hashing is probabilistic. Geometry with floating point needs EPS; exact integer orientation uses PointLL.
- Recursive DFS can stack overflow on a path-shaped graph with $N \approx 2 \cdot 10^5$.
- Never paste a template without checking its stated preconditions and the problem’s constraints.

1. Setup, Fast I/O & Debugging

1.1 Contest Boilerplate (Marwan’s Setup)
64-bit integers by default. Add the test-case loop only when the problem statement has T.

```
#include <bits/stdc++.h>
using namespace std;
```

```
#define int long long
#define M 1000000007LL
#define N 1000005
#define INF (1LL << 62)
#define endl '\n'
#define all(v) (v).begin(), (v).end()
#define yes cout << "Yes\n"
#define no cout << "No\n"
#define minus cout << "-1\n"
#define zero cout << "0\n"
```

```
template<class T>
void sort_unique(vector<T>& v) {
    sort(all(v));
    v.erase(unique(all(v)), v.end());
}
```

```
void solve() {
    // Solve logic
}
```

```
int32_t main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    solve(); // If T exists: read T, then while (T--) solve();
    return 0;
}
```

1.2 GCC Optimization Pragmas
Use only when needed. Avoid CPU-specific targets in an onsite submission unless the judge architecture is known.

```
#pragma GCC optimize("O3,unroll-loops")
// Do NOT blindly use:
// #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

1.3 Local Build Systems
Optional local tooling; not part of the onsite submission template.

```
// Linux:
{
```

```
}
// Windows:
{
    "shell_cmd": "g++ -std=c++17 -O2 -Wall \"${file}\" -o \"${file_path}/${file_base_name}.exe\" && \"${file_path}/${file_base_name}.exe\" < \"${file_path}/input.txt\" > \"${file_path}/output.txt\"",
    "working_dir": "${file_path}",
    "selector": "source.cpp"
}
```

1.4 Fast I/O (fread based)
Reads integers ~ 5× faster than cin. Do NOT mix with cin/scanf.

```
inline char gc() {
    static char buf[1 << 16];
    static size_t bc = 0, be = 0;
    if (bc ≥ be) {
        buf[0] = 0; bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++];
}

int readInt() {
    int a, c;
    while ((a = gc()) < 40) if (a == 0) return 0;
    bool minus = (a == '-');
    if (minus) a = gc();
    int res = a - '0';
    while ((c = gc()) ≥ '0' && c ≤ '9') res = res * 10 + (c - '0');
    return minus ? -res : res;
}
```

1.5 Universal Debug Macro
Prints variable names and container contents (vector, set, map, pair) to stderr.

```
#ifndef ONLINE_JUDGE
#define dbg(...) cerr << __LINE__ << ": ", __dbg(#__VA_ARGS__, __VA_ARGS__), cerr << "\n"

template<class T> void _p(T x) { cerr << x; }
template<class T, class U> void _p(pair<T, U> x) { cerr << "(" << x.first << ", " << x.second << ")"; }
template<class T> void _p(vector<T> x) { cerr << "["; int _=0; for (auto y : x) { if (_++) cerr << ", "; _p(y); } cerr << "]"; }
template<class T> void _p(set<T> x) { cerr << "["; int _=0; for (auto y : x) { if (_++) cerr << ", "; _p(y); } cerr << "]"; }
template<class T, class U> void _p(map<T, U> x) { cerr << "["; int _=0; for (auto [k, v] : x) { if (_++) cerr << ", "; _p(k); cerr << ": "; _p(v); } cerr << "]"; }
void __dbg(const char* s) {}
template<class T, class... U> void __dbg(const char* s, T x, U... t) {
    int i = 0; while (s[i] && s[i] ≠ ',') i++;
    cerr.write(s, i) << "="; _p(x);
    if constexpr (sizeof...(t)) cerr << " || ", __dbg(s + i + 1, t...);
}
#else
#define dbg(...)
#endif
```

1.6 GP Hash Table & Anti-Hash Test
Randomized splitmix64 hash reduces practical collision attacks for numeric and pair keys; no hash is mathematically collision-free.

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
```

```
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }

    size_t operator()(const pair<int,int>& p) const {
        return splitmix64((uint64_t)p.first) ^ (splitmix64((uint64_t)p.second) << 1);
    }
};

template<class K, class V>
using safe_map = gp_hash_table<K, V, custom_hash>;

template<class K>
using safe_set = gp_hash_table<K, null_type, custom_hash>;
```

1.7 Basic Node Structures (Linked List & Tree)
Snippet definitions for pointers, linked lists, and tree nodes.

```
class Node {
public:
    int data;
    Node* next;
    Node(int d) : data(d), next(nullptr) {}
};

class TreeNode {
public:
    int val;
    TreeNode *left, *right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};
```

1.8 Contest Complexity & Constants Cheat Sheet
Rough guidance only. Actual limits depend heavily on language, constants, memory, and the judge.

- $2^{10} \approx 10^3$, $2^{20} \approx 10^6$, $2^{30} \approx 10^9$, $2^{60} \approx 10^{18}$.
- $O(N!)$: usually $N \lesssim 10$.
- $O(2^N)$: usually $N \lesssim 20\text{--}25$.
- $O(N^3)$: commonly $N \lesssim 300\text{--}500$.
- $O(N^2)$: commonly $N \lesssim 2000\text{--}5000$.
- $O(N \log N)$: often fine up to $10^5\text{--}10^6$.
- Never treat 10^8 as a guaranteed one-second budget.

2. Mathematics & Basic Algebra

2.1 Series & Sums Formulas
Closed form expressions for standard summations and power series.

- Arithmetic Progression: $S_n = \frac{n}{2}(2a + (n-1)d) = \frac{n}{2}(a_1 + a_n)$
- Geometric Progression: $\sum_{i=0}^n c^i = \frac{c^{n+1}-1}{c-1}$, $\sum_{i=a}^b c^i = \frac{c^{b+1}-c^a}{c-1}$ ($c \neq 1$)
- Infinite GP: $\sum_{i=0}^\infty c^i = \frac{1}{1-c}$ ($|c| < 1$)
- Sum of first n integers: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$
- Sum of squares: $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$

- Sum of cubes: $\sum_{i=1}^n i^3 = \left[\frac{n(n+1)}{2}\right]^2$
- Sum of 4th powers: $\sum_{i=1}^n i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$
- Harmonic Numbers: $H_n = \sum_{i=1}^n \frac{1}{i} \approx \ln n + \gamma$ ($\gamma \approx 0.57721566$)
- Weighted Harmonic: $\sum_{i=1}^n iH_i = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}$

2.2 Algebraic Identities

Factorizations and expansions frequently tested in algebraic manipulations.

- $(a \pm b)^2 = a^2 \pm 2ab + b^2 \quad | \quad a^2 - b^2 = (a - b)(a + b)$
- $(a \pm b)^3 = a^3 \pm 3a^2b + 3ab^2 \pm b^3 \quad | \quad a^3 \pm b^3 = (a \pm b)(a^2 \mp ab + b^2)$
- $(a + b + c)^2 = a^2 + b^2 + c^2 + 2(ab + bc + ca)$
- $a^4 + 4b^4 = (a^2 - 2ab + 2b^2)(a^2 + 2ab + 2b^2)$ (Sophie Germain)
- $a^n - b^n = (a - b)(a^{n-1} + a^{n-2}b + \dots + b^{n-1})$
- $a^n + b^n = (a + b)(a^{n-1} - a^{n-2}b + \dots + b^{n-1})$ (for odd n)

2.3 Classical Inequalities

Inequalities used for bounds, binary searches, and proofs.

- AM-GM: $\frac{x_1+x_2+\dots+x_n}{n} \geq \sqrt[n]{x_1x_2\dots x_n}$ (equality iff all x_i equal)
- Cauchy-Schwarz: $(\sum a_i^2)(\sum b_i^2) \geq (\sum a_ib_i)^2$
- Triangle: $||a| - |b|| \leq |a \pm b| \leq |a| + |b|$
- Jensen’s: $f\left(\frac{\sum x_i}{n}\right) \leq \frac{\sum f(x_i)}{n}$ for convex f ($f'' \geq 0$)
- Bernoulli: $(1 + x)^n \geq 1 + nx$ for $n \geq 0, x > -1$
- Chebyshev: $\frac{1}{n} \sum a_ib_i \geq \left(\frac{1}{n} \sum a_i\right) \left(\frac{1}{n} \sum b_i\right)$ if sorted similarly

2.4 Probability & Expected Value

Rules of probability, linearity of expectation, and statistical bounds.

- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
- Bayes’ Theorem: $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$
- Linearity of Expectation: $E[X + Y] = E[X] + E[Y]$ (ALWAYS holds, even if X, Y are dependent!)
- Independent Variables: $E[X \cdot Y] = E[X] \cdot E[Y]$
- Variance: $\text{Var}(X) = E[X^2] - (E[X])^2, \text{Var}(aX + b) = a^2 \text{Var}(X)$
- Markov: $P(X \geq a) \leq \frac{E[X]}{a}$ ($X \geq 0, a > 0$)
- Chebyshev: $P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$

2.5 Binary Exponentiation

Computes $a^b \pmod M$ in $O(\log b)$ time.

```
int binexp(int a, int b, int m = M) {
    int res = 1 % m; a %= m;
    while (b > 0) {
        if (b & 1) res = (__int128)res * a % m;
        a = (__int128)a * a % m;
        b >>= 1;
    }
}
```

```
return res;
}
```

2.5a Safe Integer Helpers

Tiny helpers worth keeping available in almost every onsite solution.

```
long long norm_mod(long long x, long long m) {
    x %= m;
    if (x < 0) x += m;
    return x;
}
```

```
long long floor_div(long long a, long long b) {
    // b > 0
    long long q = a / b, r = a % b;
    if (r < 0) q--;
    return q;
}
```

```
long long ceil_div(long long a, long long b) {
    // b > 0
    return -floor_div(-a, b);
}
```

```
void print_il28(__int128 x) {
    if (x == 0) { cout << '0'; return; }
    if (x < 0) { cout << '-'; x = -x; }
```

```
string s;
while (x) {
    s.push_back('0' + x % 10);
    x /= 10;
}
reverse(s.begin(), s.end());
cout << s;
}
```

2.6 Matrix Exponentiation

Solves linear recurrences and counts graph paths of length K in $O(N^3 \log K)$.

```
template<size_t N>
struct Matrix {
    int mat[N][N];
    Matrix(int v = 0) {
        for (size_t i = 0; i < N; i++)
            for (size_t j = 0; j < N; j++)
                mat[i][j] = (i == j ? v : 0);
    }
    Matrix operator*(const Matrix& other) const {
        Matrix res(0);
        for (size_t i = 0; i < N; i++) {
            for (size_t k = 0; k < N; k++) {
                if (!mat[i][k]) continue;
                for (size_t j = 0; j < N; j++) {
                    res.mat[i][j] = (res.mat[i][j] + (__int128)mat[i][k] *
                        other.mat[k][j]) % M;
                }
            }
        }
        return res;
    }
};

template<size_t N>
Matrix<N> mat_expo(Matrix<N> a, int p) {
    Matrix<N> res(1);
    while (p > 0) {
        if (p & 1) res = res * a;
        a = a * a;
        p >>= 1;
    }
    return res;
}

// Fibonacci: F[n] = F[n-1] + F[n-2], F[0]=0, F[1]=1
```

```
int fib_mat(int n) {
    if (n == 0) return 0;
    Matrix<2> T;
    T.mat[0][0] = 1; T.mat[0][1] = 1;
    T.mat[1][0] = 1; T.mat[1][1] = 0;
    T = mat_expo(T, n - 1);
    return T.mat[0][0]; // F[n]
}
```

2.7 XOR Basis

Linear basis over $\mathbb{F}_2$. Supports insertion, maximum subset XOR, and representability. Use non-negative 64-bit values.

```
template<class T = long long, int LG = 63>
struct Basis {
    T b[LG] = {};
    int sz = 0;
```

```
bool insert(T x) {
    for (int bit = LG - 1; bit >= 0; bit--) {
        if ((x >> bit) & 1) == 0) continue;
        if (b[bit]) x ^= b[bit];
        else {
            b[bit] = x;
            sz++;
            return true;
        }
    }
    return false;
}
```

```
bool can(T x) const {
    for (int bit = LG - 1; bit >= 0; bit--) {
        if ((x >> bit) & 1) == 0) continue;
        if (!b[bit]) return false;
        x ^= b[bit];
    }
    return true;
}
```

```
T max_xor() const {
    T ans = 0;
    for (int bit = LG - 1; bit >= 0; bit--)
        ans = max(ans, ans ^ b[bit]);
    return ans;
}
```

2.8 Linear Combination Checking (Bézout’s)

Check if integer N can be formed by $a \cdot x + b \cdot y + c \cdot z + \dots$

- N can be represented iff $N \bmod \gcd(a_1, a_2, \dots, a_k) == 0$.
- Frobenius Coin Problem: For 2 coprime integers a, b , the largest number that CANNOT be formed is $a \cdot b - a - b$. Exactly $\frac{(a-1)(b-1)}{2}$ numbers cannot be formed.

3. Number Theory & Divisors

3.1 GCD & LCM Fundamental Properties

Mathematical identities connecting GCD, LCM, divisibility, and lattice lines.

- $\gcd(a, b) \times \text{lcm}(a, b) = |a \cdot b|$
- $\gcd(a, b) = \gcd(a, b \bmod a) = \gcd(a, b - a)$
- $\gcd(k \cdot a, k \cdot b) = k \cdot \gcd(a, b)$
- $\gcd(a, b) = 1 \iff \gcd(a + b, a \cdot b) = 1$
- If $a \mid (b \cdot c)$ and $\gcd(a, b) = d$, then $\frac{a}{d} \mid c$.

- Exponents: $\gcd(a^m - 1, a^n - 1) = a^{\gcd(m,n)} - 1$
- Fibonacci GCD: $\gcd(F_m, F_n) = F_{\gcd(m,n)}$
- Lattice segment: The number of strict lattice points between $(0, 0)$ and (a, b) is $\gcd(|a|, |b|) - 1$.

3.2 Extended Euclidean Algorithm & Diophantine

Solves $ax + by = \gcd(a, b)$ and finds one solution to $ax + by = c$ for integer x, y .

```
long long extgcd(long long a, long long b, long long& x, long long& y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return g;
}
```

```
bool find_any_solution(long long a, long long b, long long c,
                      long long& x0, long long& y0, long long& g) {
    if (a == 0 && b == 0) {
        g = 0;
        if (c == 0) { x0 = y0 = 0; return true; }
        return false;
    }
```

```
    g = extgcd(llabs(a), llabs(b), x0, y0);
    if (c % g != 0) return false;

    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    // All solutions:
    // x = x0 + k*(b/g), y = y0 - k*(a/g)
    return true;
}
```

```
g = extgcd(llabs(a), llabs(b), x0, y0);
if (c % g != 0) return false;
```

```
x0 *= c / g;
y0 *= c / g;
if (a < 0) x0 = -x0;
if (b < 0) y0 = -y0;
// All solutions:
// x = x0 + k*(b/g), y = y0 - k*(a/g)
return true;
```

3.3 Modular Inverse

Fermat's theorem for prime mod M ; Extended Euclidean for arbitrary coprime m .

```
// Prime mod: a^(m-2) mod m
int modInvPrime(int a, int m = M) { return binexp(a, m - 2, m); }
```

// Arbitrary mod (requires $\gcd(a, m) == 1$):

```
int modInv(int a, int m) {
    int x, y;
    int g = extgcd(a, m, x, y);
    if (g != 1) return -1; // No inverse exists
    return (x % m + m) % m;
}
```

3.4 Chinese Remainder Theorem (General CRT)

Merges $x \equiv a_1 \pmod{m_1}$ and $x \equiv a_2 \pmod{m_2}$. Supports non-coprime moduli.

```
pair<int, int> CRT(int a1, int m1, int a2, int m2) {
    int x, y;
    int g = extgcd(m1, m2, x, y);
    if ((a2 - a1) % g != 0) return {-1, -1}; // No solution
    int lcm = (m1 / g) * m2;
    int t = ((__int128)(a2 - a1) / g * x) % (m2 / g);
    int ans = (a1 + (__int128)m1 * t) % lcm;
    if (ans < 0) ans += lcm;
    return {ans, lcm}; // x === ans (mod lcm)
}
```

3.5 Sieve of Eratosthenes

Finds all primes up to N in $O(N \log \log N)$. Call only once or clear the vectors first.

```
vector<int> primes;
vector<bool> is_prime(N + 1, true);

void sieve() {
    fill(is_prime.begin(), is_prime.end(), true);
    is_prime[0] = is_prime[1] = false;
    primes.clear();

    for (int i = 2; i * i <= N; i++)
        if (is_prime[i])
            for (int j = i * i; j <= N; j += i)
                is_prime[j] = false;

    for (int i = 2; i <= N; i++)
        if (is_prime[i]) primes.push_back(i);
}
```

3.6 Smallest Prime Factor (SPF) & Linear Sieve

Computes SPF and primes in $O(N)$; each value factorizes in $O(\log N)$.

```
vector<int> spf(N + 1), spf_primes;
```

```
void linear_sieve() {
    spf_primes.clear();
    for (int i = 2; i <= N; i++) {
        if (!spf[i]) {
            spf[i] = i;
            spf_primes.push_back(i);
        }
        for (int p : spf_primes) {
            if (p > spf[i] || i * p > N) break;
            spf[i * p] = p;
        }
    }
}
```

```
vector<pair<int, int>> factorize_spf(int n) {
    vector<pair<int, int>> f;
    while (n > 1) {
        int p = spf[n], cnt = 0;
        while (n % p == 0) n /= p, cnt++;
        f.push_back({p, cnt});
    }
    return f;
}
```

3.7 Prime Factorization in $O(\sqrt{N})$

Good for isolated values up to about 10^{18} when trial division is acceptable. Use Pollard-Rho for harder 64-bit inputs.

```
vector<pair<long long, int>> factorize(long long n) {
    vector<pair<long long, int>> f;
    for (long long d = 2; d <= n / d; d += (d == 2 ? 1 : 2)) {
        if (n % d == 0) {
            int cnt = 0;
            while (n % d == 0) n /= d, cnt++;
            f.push_back({d, cnt});
        }
    }
    if (n > 1) f.push_back({n, 1});
    return f;
}
```

3.8 Segmented Sieve

Finds primes in $[L, R]$ for $L \geq 0$ and moderate $R - L$. The memory cost is $O(R - L)$.

```
vector<int> segmented_sieve(long long L, long long R) {
    long long lim = sqrtl((long double)R);
    vector<bool> mark(lim + 1, true);
```

```
vector<int> primes;
```

```
if (lim >= 0) mark[0] = false;
if (lim >= 1) mark[1] = false;
for (long long i = 2; i * i <= lim; i++)
    if (mark[i])
        for (long long j = i * i; j <= lim; j += i)
            mark[j] = false;
for (int i = 2; i <= lim; i++)
    if (mark[i]) primes.push_back(i);

vector<bool> ok(R - L + 1, true);
for (long long p : primes) {
    long long start = max(p * p, (L + p - 1) / p * p);
    for (long long x = start; x <= R; x += p)
        ok[x - L] = false;
}
```

```
for (long long x = L; x <= min(R, 1LL); x++)
    ok[x - L] = false;
```

```
vector<int> ans;
for (long long x = L; x <= R; x++)
    if (ok[x - L]) ans.push_back((int)x);
return ans;
}
```

3.9 Divisor Formulas

For $n = \prod p_i^{a_i}$. Keep the formulas; avoid memorizing problem-specific maximum-divisor tables.

- Number of divisors: $\tau(n) = \prod_i (a_i + 1)$.
- Sum of divisors: $\sigma(n) = \prod_i \frac{p_i^{a_i+1} - 1}{p_i - 1}$.
- Product of divisors: $\prod_{d|n} d = n^{\tau(n)/2}$.
- If $n = 2^a m$ with m odd: odd divisors $= \tau(m)$, even divisors $= a \tau(m)$.

- Sum of k -th powers: $\sigma_k(n) = \prod_i \frac{p_i^{(a_i+1)k} - 1}{p_i^k - 1}$.

- $\sum_{i=1}^N \tau(i) = \sum_{i=1}^N \left\lfloor \frac{N}{i} \right\rfloor$.

3.10 Sum of $\lfloor N/i \rfloor$ in $O(\sqrt{N})$

Groups equal values of $\lfloor N/i \rfloor$ into blocks $[l, r]$.

```
long long sum_floor_div(long long n) {
    long long ans = 0;
    for (long long l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        ans += (r - l + 1) * (n / l); // ensure answer fits in ll
    }
    return ans;
}
```

3.11 Euler's Totient Function (ETF / $\phi(n)$)

Single query in $O(\sqrt{n})$ or all $\phi(1..N)$ in $O(N \log \log N)$.

```
// Single query:
long long phi(long long n) {
    long long res = n;
    for (long long p = 2; p <= n / p; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            res -= res / p;
        }
    }
```

```
    }
}
if (n > 1) res -= res / n;
return res;
}

// Sieve for all phi[1..N]:
vector<int> phi_arr(N + 1);
void sieve_phi() {
    iota(phi_arr.begin(), phi_arr.end(), 0);
    for (int p = 2; p ≤ N; p++)
        if (phi_arr[p] == p)
            for (int x = p; x ≤ N; x += p)
                phi_arr[x] -= phi_arr[x] / p;
}

Useful facts: ∑_{d|n} ϕ(d) = n and, for n > 1, the sum of integers in [1, n]
coprime to n is nϕ(n)/2. Euler’s theorem requires gcd(a, m) = 1.

3.13 Miller-Rabin + Pollard-Rho (64-bit)
Deterministic Miller-Rabin bases below work for unsigned 64-bit integers.
Pollard-Rho needs randomized recursion to factor completely.
using u64 = uint64_t;
using u128 = __uint128_t;

u64 mul_mod(u64 a, u64 b, u64 mod) {
    return (u128)a * b % mod;
}

u64 pow_mod(u64 a, u64 e, u64 mod) {
    u64 r = 1;
    while (e) {
        if (e & 1) r = mul_mod(r, a, mod);
        a = mul_mod(a, a, mod);
        e >>= 1;
    }
    return r;
}

bool is_prime64(u64 n) {
    if (n < 2) return false;
    for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29
        ULL, 31ULL, 37ULL}) {
        if (n % p == 0) return n == p;
    }

    u64 d = n - 1;
    int s = 0;
    while ((d & 1) == 0) d >>= 1, s++;

    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL
        , 1795265022ULL}) {
        if (a % n == 0) continue;
        u64 x = pow_mod(a % n, d, n);
        if (x == 1 || x == n - 1) continue;

        bool comp = true;
        for (int r = 1; r < s; r++) {
            x = mul_mod(x, x, n);
            if (x == n - 1) { comp = false; break; }
        }
        if (comp) return false;
    }
    return true;
}

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count
    ());

u64 pollard(u64 n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
```

```
while (true) {
    u64 c = uniform_int_distribution<u64>(1, n - 1)(rng);
    u64 x = uniform_int_distribution<u64>(0, n - 1)(rng);
    u64 y = x, d = 1;

    auto f = [&](u64 v) { return (mul_mod(v, v, n) + c) % n; };

    while (d == 1) {
        x = f(x);
        y = f(f(y));
        u64 diff = x > y ? x - y : y - x;
        d = gcd(diff, n);
    }
    if (d ≠ n) return d;
}

}

void factor64(u64 n, map<u64, int>& f) {
    if (n == 1) return;
    if (is_prime64(n)) { f[n]++; return; }
    u64 d = pollard(n);
    factor64(d, f);
    factor64(n / d, f);
}

3.14 Floor Sum (AtCoder ACL)
Computes ∑_{i=0}^{n-1} ⌊ (a·i+b)/m ⌋ in O(log m) via Euclidean division.
int floor_sum(int n, int m, int a, int b) {
    int ans = 0;
    if (a ≥ m) { ans += (n - 1) * n * (a / m) / 2; a %= m; }
    if (b ≥ m) { ans += n * (b / m); b %= m; }
    int y_max = (a * n + b) / m;
    if (y_max == 0) return ans;
    ans += floor_sum(y_max, a, m, (a * n + b) % m);
    return ans;
}

3.15 Discrete Logarithm (Baby-Step Giant-Step)
Finds smallest x ≥ 0 such that a^x ≡ b (mod m) with gcd(a, m) = 1 in O(√m).
int baby_step_giant_step(int a, int b, int m) {
    a %= m; b %= m;
    if (b == 1 || m == 1) return 0;
    int n = sqrt(m) + 1;
    unordered_map<int, int> tbl;
    int cur = b;
    for (int q = 0; q < n; q++) {
        tbl[cur] = q;
        cur = (__int128)cur * a % m;
    }
    int an = binexp(a, n, m);
    cur = 1;
    for (int p = 1; p ≤ n + 1; p++) {
        cur = (__int128)cur * an % m;
        if (tbl.count(cur)) {
            int ans = p * n - tbl[cur];
            return ans;
        }
    }
    return -1;
}

3.16 Möbius Inversion Formula
μ(n) = (−1)^k if n is square-free with k prime factors, 0 if p^2 | n.
• ∑_{d|n} μ(d) = [n == 1]
• Inversion: g(n) = ∑_{d|n} f(d) ⇔ f(n) = ∑_{d|n} μ(d)g(n/d)
• Co-primality count: [gcd(i, j) == 1] = ∑_{d|gcd(i, j)} μ(d)
```

- GCD pairs: ∑_{1 ≤ i, j ≤ N} [gcd(i, j) = 1] = ∑_{d=1}^N μ(d) ⌊N/d⌋^2

3.17 Lifting The Exponent (LTE)

ν_p(x) is exponent of highest power of prime p dividing x.

- Odd Prime (p ≥ 3), p | (a − b), p ∤ a, p ∤ b: ν_p(a^n − b^n) = ν_p(a − b) + ν_p(n)
- Odd Prime (p ≥ 3), n odd, p | (a + b): ν_p(a^n + b^n) = ν_p(a + b) + ν_p(n)
- Even Prime (p = 2), a, b odd: ν_2(a^n − b^n) = ν_2(a − b) + ν_2(a + b) + ν_2(n) − 1 (for even n)

3.18 Fibonacci Properties & Fast Doubling

Computes {F_n, F_{n+1}} in O(log n) using F_{2k} = F_k(2F_{k+1} − F_k), F_{2k+1} = F_k^2 + F_{k+1}^2.

```
pair<int, int> fib_fast(int n) {
    if (n == 0) return {0, 1};
    auto [a, b] = fib_fast(n >> 1);
    int c = (a * (2 * b - a + M) % M) % M;
    int d = (a * a + b * b) % M;
    if (n & 1) return {d, (c + d) % M};
    return {c, d};
}
```

4. Combinatorics & Discrete Math

4.1 Permutations, Combinations & Modular nCr

For prime modulus M and 0 ≤ n ≤ N < M. Precompute once in O(N), query in O(1).

```
vector<int> fact(N + 1), ifact(N + 1);
```

```
void precompute_nCr() {
    fact[0] = 1;
    for (int i = 1; i ≤ N; i++) fact[i] = fact[i - 1] * i % M;

    ifact[N] = binexp(fact[N], M - 2, M);
    for (int i = N; i ≥ 1; i--) ifact[i - 1] = ifact[i] * i % M;
}
```

```
int nCr(int n, int r) {
    if (r < 0 || r > n || n > N) return 0;
    return fact[n] * ifact[r] % M * ifact[n - r] % M;
}
```

```
int nPr(int n, int r) {
    if (r < 0 || r > n || n > N) return 0;
    return fact[n] * ifact[n - r] % M;
}
```

4.2 Binomial Coefficient Properties

Core identities for manipulating combinations in CP math problems.

- Symmetry: $\binom{n}{k} = \binom{n}{n-k}$
- Pascal’s Rule: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$
- Absorption (Committee): $k\binom{n}{k} = n\binom{n-1}{k-1}$
- Total Subsets: $\sum_{k=0}^n \binom{n}{k} = 2^n$
- Alternating Sum: $\sum_{k=0}^n (-1)^k \binom{n}{k} = 0$
- Even/Odd Sums: $\sum_{k \text{ even}} \binom{n}{k} = \sum_{k \text{ odd}} \binom{n}{k} = 2^{n-1}$
- Vandermonde’s Convolution: $\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$
- Sum of Squares: $\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$

- Weighted Sum: $\sum_{k=1}^n k \binom{n}{k} = n \cdot 2^{n-1}$

4.3 Hockey-Stick Identity (Pyramid Formula for nCr)

Summing along a diagonal or column in Pascal's triangle.

- Column Sum: $\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$
- Diagonal Sum: $\sum_{i=0}^k \binom{n+i}{i} = \binom{n+k+1}{k}$
- Pascal's Pyramid (3D / Multinomial): $\binom{n}{i,j,k} = \frac{n!}{i!j!k!}$ ($i + j + k = n$) Each entry is the sum of the 3 adjacent entries in the layer above: $\binom{n}{i,j,k} = \binom{n-1}{i-1,j,k} + \binom{n-1}{i,j-1,k} + \binom{n-1}{i,j,k-1}$

4.4 Stars and Bars (Ballot Problems)

Distributing n identical objects into k distinct containers.

- Non-negative integers ($x_i \geq 0$): $\binom{n+k-1}{k-1}$
- Strictly positive integers ($x_i \geq 1$): $\binom{n-1}{k-1}$
- Lower bounded ($x_i \geq c_i$): Let $n' = n - \sum c_i$; ways $= \binom{n'+k-1}{k-1}$

4.5 Catalan Numbers (Formulas & CP Explanations)

Sequence: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, ...

- Explicit Formula: $C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$
- Recurrence: $C_0 = 1$, $C_{n+1} = \sum_{i=0}^n C_i C_{n-i} = \frac{2(2n+1)}{n+2} C_n$
- CP Interpretation & Applications:
 1. Balanced Parentheses: Number of valid expressions with n pairs of '(' and ')'.
 2. Dyck Paths: Monotonic grid walks from $(0,0)$ to (n,n) that never cross strictly above the diagonal $y = x$.
 3. Polygon Triangulation: Ways to divide a convex polygon of $n + 2$ vertices into n triangles with non-intersecting diagonals.
 4. Full Binary Trees: Number of distinct rooted binary trees with $n + 1$ leaves (n internal nodes).
 5. Non-intersecting Chords: Ways to pair $2n$ points on a circle with n non-intersecting chords.
 6. Stack Permutations: Permutations of length n that can be sorted using a single stack (avoiding pattern 231).

```
int catalan(int n) {
    return nCr(2 * n, n) * modInvPrime(n + 1) % M;
}
```

4.6 Derangements $D(n)$

Permutations where no element appears in its original position ($\pi(i) \neq i$).

- Sequence: 0, 1, 2, 9, 44, 265, 1854, 14833, ...
- Recurrence: $D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n$
- Closed form: $D(n) = n! \sum_{k=0}^n \frac{(-1)^k}{k!} \approx \left\lfloor \frac{n!}{e} \right\rfloor$

4.7 Stirling Numbers of 1st and 2nd Kind

Cycle arrangements and set partitions.

- 1st Kind $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right]$: Permutations of n elements with exactly k disjoint cycles. $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right] = \left[\begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right] + (n-1) \left[\begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right]$, $\sum_{k=0}^n \left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right] = n!$
- 2nd Kind $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$: Ways to partition n distinct elements into k non-empty sets. $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} = \left\{ \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\} + k \left\{ \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\}$, $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^n$
- Bell Numbers $B(n) = \sum_{k=0}^n \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$: Total ways to partition n distinct elements.

4.8 Lucas' Theorem

Computes $\binom{n}{r} \bmod p$ for large n, r when p is a small prime. The small combinations must also be taken modulo p .

```
struct Lucas {
    long long p;
    vector<long long> fact, ifact;

    Lucas(long long p) : p(p), fact(p), ifact(p) {
        fact[0] = 1;
        for (int i = 1; i < p; i++) fact[i] = fact[i-1] * i % p;
        ifact[p-1] = binexp(fact[p-1], p-2, p);
        for (int i = p-1; i >= 1; i--)
            ifact[i-1] = ifact[i] * i % p;
    }

    long long Csmall(long long n, long long r) {
        if (r < 0 || r > n) return 0;
        return fact[n] * ifact[r] % p * ifact[n-r] % p;
    }

    long long C(long long n, long long r) {
        if (r < 0 || r > n) return 0;
        long long ans = 1;
        while (n || r) {
            long long ni = n % p, ri = r % p;
            if (ri > ni) return 0;
            ans = ans * Csmall(ni, ri) % p;
            n /= p;
            r /= p;
        }
        return ans;
    }
};
```

4.9 Inclusion-Exclusion Principle

Counts unions and intersections by alternating sums of subset intersections.

```
// Count elements satisfying none of n conditions
template<typename Func>
int inclusion_exclusion(int n, Func f) {
    int ans = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        int sign = (__builtin_popcount(mask) & 1) ? -1 : 1;
        ans += sign * f(mask);
    }
}
```

```
}
return ans;
}
```

4.10 Burnside's Lemma (Colorings Under Symmetry)

Number of orbits under group actions: $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$ where $|X^g| = k^{c(g)}$.

- Necklaces with n beads and k colors under rotation: $\frac{1}{n} \sum_{i=0}^{n-1} k^{\gcd(i,n)}$
- Bracelets (rotation + reflection): Add reflection symmetries to group size.

5. Data Structures

5.1 Policy Based Data Structures (PBDS Ordered Set)

Supports `find_by_order(k)` and `order_of_key(x)` in $O(\log N)$. Multiset uses `pair<T, int>` to avoid UB.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <class T, class Cmp = less<T>>
using ordered_set = tree<T, null_type, Cmp, rb_tree_tag,
tree_order_statistics_node_update>;

// Safe Multiset without UB:
ordered_set<pair<int, int>> ost;
int timer = 0;
void insert_val(int x) { ost.insert({x, ++timer}); }
void erase_val(int x) {
    auto it = ost.lower_bound({x, 0});
    if (it != ost.end() && it->first == x) ost.erase(it);
}
int order_of(int x) { return ost.order_of_key({x, 0}); } // count < x
int kth_element(int k) { return ost.find_by_order(k)->first; } // 0-indexed
```

5.2 Disjoint Set Union (DSU)

Standard DSU with path compression and union by size. Tracks component count and sizes.

```
struct DSU {
    vector<int> p, sz;
    int comps;
    DSU(int n) : p(n+1), sz(n+1, 1), comps(n) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        return p[x] == x ? x : p[x] = find(p[x]);
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        comps--;
        return true;
    }
    int size(int x) { return sz[find(x)]; }
};
```

5.3 Rollback DSU (Undoable Disjoint Sets)

No path compression; union by rank only. Used for offline dynamic connectivity and divide-and-conquer.

```
struct RollbackDSU {
```

```

vector<int> p, sz;
vector<pair<int, int>> history;
RollbackDSU(int n) : p(n + 1), sz(n + 1, 1) {
    iota(p.begin(), p.end(), 0);
}
int find(int x) {
    while (x != p[x]) x = p[x];
    return x;
}
bool unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b);
    history.push_back({b, sz[a]});
    p[b] = a;
    sz[a] += sz[b];
    return true;
}
int time() { return history.size(); }
void rollback(int t) {
    while ((int)history.size() > t) {
        auto [b, old_sz_a] = history.back(); history.pop_back();
        int a = p[b];
        sz[a] = old_sz_a;
        p[b] = b;
    }
}
};

```

5.4 Fenwick Tree (Binary Indexed Tree)

1-indexed. Point add, prefix/range sum in $O(\log N)$, and k -th prefix search when all stored values are non-negative.

```

struct Fenwick {
    int n, pw;
    vector<int> tree;

    Fenwick(int n) : n(n), tree(n + 1, 0) {
        pw = 1;
        while ((pw << 1) <= n) pw <= 1;
    }

    void add(int i, int delta) {
        for (; i <= n; i += i & -i) tree[i] += delta;
    }

    int sum(int i) const {
        int ans = 0;
        for (; i > 0; i -= i & -i) ans += tree[i];
        return ans;
    }

    int query(int l, int r) const {
        return sum(r) - sum(l - 1);
    }

    int find_kth(int k) const {
        // Smallest idx with prefix sum ≥ k.
        // Requires 1 ≤ k ≤ total sum and non-negative frequencies.
        int idx = 0;
        for (int step = pw; step; step >= 1) {
            if (idx + step <= n && tree[idx + step] < k) {
                idx += step;
                k -= tree[idx];
            }
        }
        return idx + 1;
    }
};

```

5.5 Range Update & Range Query Fenwick (2-BIT Trick)

Supports range add and range sum using $p(i) = i \cdot \text{bit}_1(i) - \text{bit}_2(i)$.

```

struct RangeFenwick {
    int n;
    vector<int> b1, b2;
    RangeFenwick(int n) : n(n), b1(n + 2, 0), b2(n + 2, 0) {}
    void add_internal(vector<int>& b, int i, int v) {
        for (; i <= n; i += i & -i) b[i] += v;
    }
    void range_add(int l, int r, int v) {
        add_internal(b1, l, v); add_internal(b1, r + 1, -v);
        add_internal(b2, l, v * (1 - 1)); add_internal(b2, r + 1, -v * r);
    }
    int prefix_sum(int i) {
        int s1 = 0, s2 = 0;
        for (int x = i; x > 0; x -= x & -x) s1 += b1[x], s2 += b2[x];
        return s1 * i - s2;
    }
    int range_sum(int l, int r) { return prefix_sum(r) - prefix_sum(l - 1); }
};

```

5.6 2D Fenwick Tree

Point update and 2D prefix/submatrix sum in $O(\log N \log M)$.

```

struct Fenwick2D {
    int n, m;
    vector<vector<int>> tree;
    Fenwick2D(int n, int m) : n(n), m(m), tree(n + 1, vector<int>(m + 1, 0)) {}
    void add(int x, int y, int v) {
        for (int i = x; i <= n; i += i & -i)
            for (int j = y; j <= m; j += j & -j) tree[i][j] += v;
    }
    int query(int x, int y) {
        int sum = 0;
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j) sum += tree[i][j];
        return sum;
    }
    int query(int x1, int y1, int x2, int y2) {
        return query(x2, y2) - query(x1 - 1, y2) - query(x2, y1 - 1) +
            query(x1 - 1, y1 - 1);
    }
};

```

5.7 Segment Tree (User's Point Update, Range Sum)

Zero-indexed array implementation matching user snippet.

```

class SGTree {
public:
    vector<int> seg;
    SGTree(int n) { seg.resize(4 * n + 1, 0); }
    void build(int idx, int l, int r, vector<int>& v) {
        if (l == r) { seg[idx] = v[l]; return; }
        int mid = (l + r) / 2;
        build(2 * idx + 1, l, mid, v);
        build(2 * idx + 2, mid + 1, r, v);
        seg[idx] = seg[2 * idx + 1] + seg[2 * idx + 2];
    }
    void update(int idx, int l, int r, int pos, int val) {
        if (l == r) { seg[idx] = val; return; }
        int mid = (l + r) / 2;
        if (pos <= mid) update(2 * idx + 1, l, mid, pos, val);
        else update(2 * idx + 2, mid + 1, r, pos, val);
        seg[idx] = seg[2 * idx + 1] + seg[2 * idx + 2];
    }
    int query(int idx, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return seg[idx];
        int mid = (l + r) / 2;
        return query(2 * idx + 1, l, mid, ql, qr) + query(2 * idx + 2,
            mid + 1, r, ql, qr);
    }
};

```

```
};
```

5.8 Lazy Segment Tree (Range Add & Range Sum)

Range add + range sum in $O(\log N)$. For min/max, the node update formula must be changed.

```

struct LazySeg {
    int n;
    vector<int> tree, lazy;

    LazySeg(int n) : n(n), tree(4 * n + 4), lazy(4 * n + 4) {}

    void push(int node, int l, int r) {
        if (!lazy[node]) return;
        tree[node] += (r - l + 1) * lazy[node];

        if (l != r) {
            lazy[node * 2] += lazy[node];
            lazy[node * 2 + 1] += lazy[node];
        }
        lazy[node] = 0;
    }

    void add(int node, int l, int r, int ql, int qr, int val) {
        push(node, l, r);
        if (qr < l || r < ql) return;

        if (ql <= l && r <= qr) {
            lazy[node] += val;
            push(node, l, r);
            return;
        }

        int mid = (l + r) / 2;
        add(node * 2, l, mid, ql, qr, val);
        add(node * 2 + 1, mid + 1, r, ql, qr, val);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    int query(int node, int l, int r, int ql, int qr) {
        push(node, l, r);
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[node];

        int mid = (l + r) / 2;
        return query(node * 2, l, mid, ql, qr) +
            query(node * 2 + 1, mid + 1, r, ql, qr);
    }
};

```

5.9 Persistent Segment Tree (K-th Smallest in Range)

Typical use: build one root per prefix after coordinate compression. Node indices are 32-bit; sums are 64-bit.

```

struct PST {
    struct Node {
        int lc = 0, rc = 0;
        long long sum = 0;
    };

    vector<Node> tr;
    vector<int> root;
    int nodes = 0;

    PST(int versions = 0) {
        tr.resize(1); // node 0 = null
        root.assign(versions + 1, 0);
    }

    int update(int prev, int l, int r, int pos, int val = 1) {
        int cur = ++nodes;
        if ((int)tr.size() <= cur) tr.push_back({});
        tr[cur] = tr[prev];
    }
};

```

```

    tr[cur].sum += val;

    if (l == r) return cur;

    int mid = (l + r) / 2;
    if (pos ≤ mid)
        tr[cur].lc = update(tr[prev].lc, l, mid, pos, val);
    else
        tr[cur].rc = update(tr[prev].rc, mid + 1, r, pos, val);

    return cur;
}

int kth(int left_root, int right_root, int l, int r, long long k)
{
    if (l == r) return l;

    long long left_count =
        tr[tr[right_root].lc].sum - tr[tr[left_root].lc].sum;

    int mid = (l + r) / 2;
    if (k ≤ left_count)
        return kth(tr[left_root].lc, tr[right_root].lc, l, mid, k);

    return kth(tr[left_root].rc, tr[right_root].rc, mid + 1, r,
        k - left_count);
}
};

```

5.10 Sparse Table (Static Range Minimum)

Idempotent RMQ. Preprocessing $O(N \log N)$, query $O(1)$. Replace min with max for maximum queries.

```

struct SparseMin {
    int n, K;
    vector<vector<int>> st;

    SparseMin(const vector<int>& a) : n(a.size()) {
        if (n == 0) return;
        K = __lg(n) + 1;
        st.assign(K, vector<int>(n));
        st[0] = a;

        for (int j = 1; j < K; j++)
            for (int i = 0; i + (1 << j) ≤ n; i++)
                st[j][i] = min(st[j - 1][i],
                    st[j - 1][i + (1 << (j - 1))]);

        int query(int l, int r) const {
            int j = __lg(r - l + 1);
            return min(st[j][l], st[j][r - (1 << j) + 1]);
        }
    }
};

```

5.11 Disjoint Sparse Table (Range Sum)

Non-idempotent range sum in $O(1)$ after $O(N \log N)$ preprocessing. This implementation is sum-only.

```

struct DisjointSparseSum {
    int n, levels;
    vector<vector<int>> st;

    DisjointSparseSum(const vector<int>& a) {
        n = a.size();
        if (!n) { levels = 0; return; }

        int m = 1;
        while (m < n) m <= 1;
        levels = __lg(m) + 1;
        st.assign(levels, vector<int>(m));

        vector<int> b = a;

```

```

        b.resize(m, 0);
        st[0] = b;

        for (int h = 1; h < levels; h++) {
            int half = 1 << (h - 1);
            int block = half << 1;

            for (int start = 0; start < m; start += block) {
                int mid = start + half;
                if (mid > m) break;

                st[h][mid - 1] = b[mid - 1];
                for (int i = mid - 2; i ≥ start; i--)
                    st[h][i] = st[h][i + 1] + b[i];

                if (mid < m) {
                    st[h][mid] = b[mid];
                    for (int i = mid + 1; i < min(m, start + block); i++)
                        st[h][i] = st[h][i - 1] + b[i];
                }
            }
        }

        int query(int l, int r) const {
            if (l == r) return st[0][l];
            int h = __lg(1 ^ r) + 1;
            return st[h][l] + st[h][r];
        }
    };
};

```

5.12 Alphabet Trie & Binary Trie (Max XOR)

String trie for lowercase letters; binary trie for non-negative 63-bit integers.

```

// Lowercase alphabet trie:
struct TrieNode {
    TrieNode* nxt[26] = {};
    bool is_end = false;
};

void insert_str(TrieNode* root, const string& s) {
    TrieNode* cur = root;
    for (char c : s) {
        int x = c - 'a';
        if (!cur->nxt[x]) cur->nxt[x] = new TrieNode();
        cur = cur->nxt[x];
    }
    cur->is_end = true;
}

```

// Max XOR against a stored non-negative value:

```

struct BitTrie {
    struct Node {
        int nxt[2] = {-1, -1};
        int cnt = 0;
    };

    vector<Node> tr;

    BitTrie() { tr.push_back({}); }

```

```

    void insert(long long x) {
        int cur = 0;
        tr[cur].cnt++;

        for (int bit = 62; bit ≥ 0; bit--) {
            int b = (x >> bit) & 1;
            if (tr[cur].nxt[b] == -1) {
                tr[cur].nxt[b] = tr.size();
                tr.push_back({});
            }
            cur = tr[cur].nxt[b];
        }
    }
};

```

```

        tr[cur].cnt++;
    }

    long long max_xor(long long x) const {
        if (!tr[0].cnt) return 0; // no stored value
        int cur = 0;
        long long ans = 0;

        for (int bit = 62; bit ≥ 0; bit--) {
            int b = (x >> bit) & 1;
            int want = b ^ 1;

            if (tr[cur].nxt[want] ≠ -1 &&
                tr[tr[cur].nxt[want]].cnt > 0) {
                ans |= 1LL << bit;
                cur = tr[cur].nxt[want];
            } else {
                cur = tr[cur].nxt[b];
            }
        }
        return ans;
    }
};

```

5.13 Monotonic Stack & Monotonic Queue

Nearest Smaller to Left in $O(N)$; Sliding window maximum in $O(1)$ amortized.

```

// Nearest Smaller to Left (Values):
vector<int> nearest_smaller_left(const vector<int>& a) {
    int n = a.size();
    vector<int> ans(n, -1);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && st.top() ≥ a[i]) st.pop();
        if (!st.empty()) ans[i] = st.top();
        st.push(a[i]);
    }
    return ans;
}

// Sliding Window Maximum (Window size K):
vector<int> sliding_max(const vector<int>& a, int k) {
    deque<int> dq;
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && dq.front() ≤ i - k) dq.pop_front();
        while (!dq.empty() && a[dq.back()] ≤ a[i]) dq.pop_back();
        dq.push_back(i);
        if (i ≥ k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}

```

5.14 Mo's Algorithm (Snake Order)

Offline range queries in $O((N + Q)\sqrt{N})$. Alternating sorting cuts pointer moves.

```

struct Query {
    int l, r, id;
};

void solve_mo(int n, vector<Query>& queries) {
    int B = max(1LL, (int)sqrt(n));
    sort(queries.begin(), queries.end(), [&](const Query& a, const
        Query& b) {
        if (a.l / B ≠ b.l / B) return a.l / B < b.l / B;
        return (a.l / B) & 1 ? a.r < b.r : a.r > b.r;
    });
    int curL = 0, curR = -1;
    auto add = [&](int idx) { /* include a[idx] */ };
    auto remove = [&](int idx) { /* remove a[idx] */ };
    for (auto &q : queries) {
        while (curL > q.l) add(--curL);

```

```

while (curR < q.r) add(++curR);
while (curL < q.l) remove(curL++);
while (curR > q.r) remove(curR--);
// ans[q.id] = current_answer;
}
}

```

5.15 Small-to-Large Merging

When merging data sets between components, always merge the smaller into the larger for $O(N \log N)$ total time.

```

void merge_sets(vector<int>& comp_a, vector<int>& comp_b) {
    if (comp_a.size() < comp_b.size()) swap(comp_a, comp_b);
    for (int val : comp_b) comp_a.push_back(val);
    comp_b.clear();
}

```

5.16 Coordinate Compression

Maps large coordinate values to $[0, K - 1]$ while preserving order.

```

vector<int> vals = a;
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());
for (int &x : a) x = lower_bound(vals.begin(), vals.end(), x) - vals
    .begin();

```

6. String Algorithms

6.1 Double Polynomial Rolling Hash

Static substring hashing in $O(1)$ with two moduli. Double hashing greatly reduces collision probability; it does not make collisions impossible.

```

struct StringHash {
    int n;
    static const int mod1 = 1000000007;
    static const int mod2 = 1000000009;
    static const int base1 = 313;
    static const int base2 = 317;

    vector<int> h1, h2, p1, p2;

    StringHash(const string& s)
        : n(s.size()), h1(n + 1), h2(n + 1),
          p1(n + 1, 1), p2(n + 1, 1) {
        for (int i = 0; i < n; i++) {
            p1[i + 1] = (__int128)p1[i] * base1 % mod1;
            p2[i + 1] = (__int128)p2[i] * base2 % mod2;
            h1[i + 1] = ((__int128)h1[i] * base1 + (unsigned char)s[i]) %
                mod1;
            h2[i + 1] = ((__int128)h2[i] * base2 + (unsigned char)s[i]) %
                mod2;
        }
    }

    pair<int, int> get_hash(int l, int r) const {
        if (l > r) return {0, 0};

        int len = r - l + 1;
        int x1 = (h1[r + 1] -
            (__int128)h1[l] * p1[len]) % mod1;
        int x2 = (h2[r + 1] -
            (__int128)h2[l] * p2[len]) % mod2;

        if (x1 < 0) x1 += mod1;
        if (x2 < 0) x2 += mod2;
        return {x1, x2};
    }
};

```

6.2 Dynamic String Hash (Point Update Segment Tree)

Single-character replacement and substring hash in $O(\log N)$. Requires a non-empty string.

```

struct DynamicHash {

```

```

int n;
string s;
vector<int> h, pw;
static const int base = 313;
static const int mod = 1000000007;

```

```

DynamicHash(string str) : n(str.size()), s(move(str)),
    h(4 * max(1LL, n) + 4), pw(n + 1, 1) {
    for (int i = 1; i ≤ n; i++)
        pw[i] = (__int128)pw[i - 1] * base % mod;
    if (n) build(1, 0, n - 1);
}

```

```

void build(int node, int l, int r) {
    if (l == r) { h[node] = s[l]; return; }
    int mid = (l + r) / 2;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    h[node] = ((__int128)h[node * 2] * pw[r - mid] +
        h[node * 2 + 1]) % mod;
}

```

```

void update(int node, int l, int r, int pos, char c) {
    if (l == r) { h[node] = s[pos] = c; return; }
    int mid = (l + r) / 2;
    if (pos ≤ mid) update(node * 2, l, mid, pos, c);
    else update(node * 2 + 1, mid + 1, r, pos, c);
    h[node] = ((__int128)h[node * 2] * pw[r - mid] +
        h[node * 2 + 1]) % mod;
}

```

```

int query(int node, int l, int r, int ql, int qr) const {
    if (ql ≤ l && r ≤ qr) return h[node];

    int mid = (l + r) / 2;
    if (qr ≤ mid) return query(node * 2, l, mid, ql, qr);
    if (ql > mid) return query(node * 2 + 1, mid + 1, r, ql, qr);

    int left_h = query(node * 2, l, mid, ql, qr);
    int right_h = query(node * 2 + 1, mid + 1, r, ql, qr);
    int right_len = min(r, qr) - mid;

    return ((__int128)left_h * pw[right_len] + right_h) % mod;
}

```

6.3 Prefix Function (KMP) & Pattern Matching

$\pi[i]$ is the longest proper prefix of $s[0..i]$ that is also its suffix. Direct KMP avoids separator-character issues.

```

vector<int> prefix_function(const string& s) {
    int n = s.size();
    vector<int> pi(n);

    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] ≠ s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

```

```

vector<int> kmp_search(const string& text, const string& pat) {
    vector<int> ans;
    if (pat.empty()) return ans;

```

```

    vector<int> pi = prefix_function(pat);
    int j = 0;

```

```

    for (int i = 0; i < (int)text.size(); i++) {
        while (j > 0 && text[i] ≠ pat[j]) j = pi[j - 1];

```

```

        if (text[i] == pat[j]) j++;

        if (j == (int)pat.size()) {
            ans.push_back(i - j + 1);
            j = pi[j - 1];
        }
    }
    return ans;
}

```

6.4 Z-Algorithm

$Z[i]$ is the length of the longest common prefix of s and $s[i..n - 1]$ in $O(N)$ time.

```

vector<int> z_function(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i ≤ r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
    }
    return z;
}

```

6.5 Manacher's Algorithm (Palindromes in $O(N)$)

Transforms the string with separators. Here $p[i]$ is the palindrome radius in the transformed string, and $\max p[i]$ equals the longest palindrome length in the original string.

```

vector<int> manacher(const string& s) {
    string t = "@";
    for (char c : s) {
        t += '#';
        t += c;
    }
    t += "$";

    int n = t.size();
    vector<int> p(n);
    int l = 0, r = 0;

    for (int i = 1; i < n - 1; i++) {
        int mirror = l + r - i;
        if (i < r) p[i] = min(r - i, p[mirror]);

        while (t[i + p[i] + 1] == t[i - p[i] - 1])
            p[i]++;

        if (i + p[i] > r) {
            l = i - p[i];
            r = i + p[i];
        }
    }
    return p;
}

```

```

bool is_palindrome(const vector<int>& p, int l, int r) {
    if (l > r) return true;
    int center = l + r + 2; // transformed index
    int len = r - l + 1;
    return p[center] ≥ len;
}

```

```

// Longest palindrome length:
int longest_palindrome(const vector<int>& p) {
    return *max_element(p.begin(), p.end());
}

```

6.6 Suffix Array & LCP ($O(N \log N)$)

Prefix-doubling suffix array with counting sort. Returns suffix array and Kasai LCP in $O(N \log N)$.

```

struct SuffixArray {
    int n;
    vector<int> sa, lcp, rank;

    SuffixArray(const string& s) {
        n = s.size();
        int total = n + 1; // extra sentinel

        vector<int> a(total);
        for (int i = 0; i < n; i++)
            a[i] = (unsigned char)s[i] + 1;
        a[n] = 0;

        sa.resize(total);
        rank.resize(total);

        int classes = max(257, N);
        vector<int> cnt(classes), sa2(total), nrank(total);

        for (int x : a) cnt[x]++;
        for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
        for (int i = total - 1; i ≥ 0; i--) sa[--cnt[a[i]]] = i;

        rank[sa[0]] = 0;
        int c = 1;
        for (int i = 1; i < total; i++) {
            if (a[sa[i]] ≠ a[sa[i - 1]]) c++;
            rank[sa[i]] = c - 1;
        }

        for (int len = 1; len < total; len ≤ 1) {
            for (int i = 0; i < total; i++)
                sa2[i] = sa[i] - len < 0 ? sa[i] - len + total : sa[i] - len;

            fill(cnt.begin(), cnt.begin() + c, 0);
            for (int i = 0; i < total; i++) cnt[rank[sa2[i]]]++;
            for (int i = 1; i < c; i++) cnt[i] += cnt[i - 1];
            for (int i = total - 1; i ≥ 0; i--)
                sa[--cnt[rank[sa2[i]]]] = sa2[i];

            nrank[sa[0]] = 0;
            int nc = 1;
            for (int i = 1; i < total; i++) {
                pair<int, int> cur = {
                    rank[sa[i]],
                    rank[(sa[i] + len) % N]
                };
                pair<int, int> prv = {
                    rank[sa[i - 1]],
                    rank[(sa[i - 1] + len) % N]
                };
                if (cur ≠ prv) nc++;
                nrank[sa[i]] = nc - 1;
            }

            rank.swap(nrank);
            c = nc;
            if (c == total) break;
        }

        // Remove sentinel.
        sa.erase(sa.begin());

        rank.assign(n, 0);
        for (int i = 0; i < n; i++) rank[sa[i]] = i;

        lcp.assign(n, 0);
        for (int i = 0, h = 0; i < n; i++) {
            int rnk = rank[i];
            if (rnk == 0) continue;

```

```

        int j = sa[rnk - 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h])
            h++;

        lcp[rnk] = h;
        if (h) h--;
    }

    // Number of distinct substrings:
    // n*(n+1)/2 - sum(lcp).
};

```

6.7 Aho-Corasick Automaton

Multi-pattern string matching in $O(\text{Text} + \sum |\text{Patterns}|)$. Builds BFS failure links.

```

struct AhoCorasick {
    struct Node { int nxt[26] = {}, fail = 0, match_cnt = 0; };
    vector<Node> t;
    AhoCorasick() { t.push_back({}); }
    void insert(const string& s) {
        int cur = 0;
        for (char c : s) {
            int x = c - 'a';
            if (!t[cur].nxt[x]) { t[cur].nxt[x] = t.size(); t.push_back({}); }
            cur = t[cur].nxt[x];
        }
        t[cur].match_cnt++;
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < 26; c++) if (t[0].nxt[c]) q.push(t[0].nxt[c]);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int c = 0; c < 26; c++) {
                if (t[u].nxt[c]) {
                    t[t[u].nxt[c]].fail = t[t[u].fail].nxt[c];
                    t[t[u].nxt[c]].match_cnt += t[t[t[u].nxt[c]].fail].match_cnt;
                    q.push(t[u].nxt[c]);
                } else {
                    t[u].nxt[c] = t[t[u].fail].nxt[c];
                }
            }
        }
    }
};

```

6.8a Suffix Automaton

Compact structure for all substrings. Build in $O(N)$; useful for distinct substring counts, occurrence information, and longest common substring variants.

```

struct SuffixAutomaton {
    struct State {
        int next[26];
        int link = -1, len = 0;
        State() { fill(next, next + 26, -1); }
    };

    vector<State> st;
    int last = 0;

    SuffixAutomaton(int n = 0) {
        st.reserve(2 * max(1, n));
        st.push_back(State());
    }

    void extend(char ch) {
        int c = ch - 'a';
        int cur = st.size();

```

```

        st.push_back(State());
        st[cur].len = st[last].len + 1;

        int p = last;
        while (p ≠ -1 && st[p].next[c] == -1) {
            st[p].next[c] = cur;
            p = st[p].link;
        }

        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];

            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = st.size();
                st.push_back(st[q]);
                st[clone].len = st[p].len + 1;

                while (p ≠ -1 && st[p].next[c] == q) {
                    st[p].next[c] = clone;
                    p = st[p].link;
                }

                st[q].link = st[cur].link = clone;
            }

            last = cur;
        }

        void build(const string& s) {
            for (char c : s) extend(c);
        }

        long long distinct_substrings() const {
            long long ans = 0;
            for (int v = 1; v < (int)st.size(); v++)
                ans += st[v].len - st[st[v].link].len;
            return ans;
        }
};

```

6.8 Booth's Algorithm (Minimum Lexicographical Rotation)

Finds starting index of lexicographically smallest cyclic shift of s in $O(N)$ time.

```

int min_cyclic_string(string s) {
    s += s;
    int n = s.size(), i = 0, ans = 0;
    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;
        while (j < n && s[k] ≤ s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i ≤ k) i += j - k;
    }
    return ans;
}

```

6.9 String Utility Helpers

Fast conversions, tokenization, and character properties.

```

// Tokenize by whitespace:
vector<string> split(const string& s) {
    stringstream ss(s); string w;
    vector<string> res;
    while (ss >> w) res.push_back(w);
    return res;
}

```

```

}
// String ↔ Number: stoi(s), stoll(s), stod(s), to_string(x)
// Char tests: islower(c), isupper(c), isdigit(c), isalpha(c),
//              tolower(c), toupper(c)

```

7. Graph Theory

7.1 Graph Traversals (DFS, BFS, 0-1 BFS, Multi-Source)

Unweighted BFS is $O(V + E)$; 0-1 BFS is $O(V + E)$ for edge weights exactly 0 or 1.

// Unweighted BFS from one source:

```

vector<int> bfs(int src, const vector<vector<int>>& g) {
    int n = g.size() - 1;
    vector<int> dist(n + 1, -1);
    queue<int> q;
    q.push(src);
    dist[src] = 0;

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}

```

// Multi-source BFS:

```

vector<int> multi_bfs(const vector<int>& sources,
                    const vector<vector<int>>& g) {
    int n = g.size() - 1;
    vector<int> dist(n + 1, -1);
    queue<int> q;

    for (int s : sources)
        if (dist[s] == -1) dist[s] = 0, q.push(s);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}

```

// 0-1 BFS:

```

vector<int> bfs01(int src, int n,
                const vector<vector<pair<int,int>>>& g) {
    vector<int> dist(n + 1, INF);
    deque<int> dq;
    dist[src] = 0;
    dq.push_front(src);

    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
    return dist;
}

```

7.2 Dijkstra's Algorithm (Single Source Shortest Path)

Finds shortest paths with non-negative edge weights in $O((V + E) \log V)$.

```

vector<int> dijkstra(int s, int n, const vector<vector<pair<int, int>>& g) {
    vector<int> dist(n + 1, INF);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<
        pair<int, int>>> pq;
    dist[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

```

7.3 Bellman-Ford & Negative Cycle Detection

Shortest paths from one source with negative edges in $O(VE)$. The negative-cycle helper detects a cycle anywhere in the graph.

```

struct Edge {
    int u, v;
    long long w;
};

vector<long long> bellman_ford(int src, int n,
                             const vector<Edge>& edges,
                             bool& neg_cycle) {
    vector<long long> dist(n + 1, INF);
    dist[src] = 0;
    neg_cycle = false;

    for (int it = 1; it ≤ n - 1; it++) {
        bool changed = false;
        for (auto e : edges) {
            if (dist[e.u] == INF) continue;
            if (dist[e.v] > dist[e.u] + e.w) {
                dist[e.v] = dist[e.u] + e.w;
                changed = true;
            }
        }
        if (!changed) break;
    }

    for (auto e : edges) {
        if (dist[e.u] != INF && dist[e.v] > dist[e.u] + e.w) {
            neg_cycle = true;
            break;
        }
    }
    return dist;
}

```

// Detect and return one negative cycle anywhere in the graph.

// Set all dist = 0: equivalent to adding a super-source.

```

bool find_negative_cycle(int n, const vector<Edge>& edges,
                        vector<int>& cycle) {
    vector<long long> dist(n + 1, 0);
    vector<int> parent(n + 1, -1);
    int x = -1;

    for (int it = 1; it ≤ n; it++) {
        x = -1;
        for (auto e : edges) {
            if (dist[e.v] > dist[e.u] + e.w) {
                dist[e.v] = dist[e.u] + e.w;
                parent[e.v] = e.u;
            }
        }
    }
    if (parent[x] != -1) {
        cycle.push_back(x);
        while (parent[x] != -1) {
            cycle.push_back(parent[x]);
            x = parent[x];
        }
        return true;
    }
    return false;
}

```

```

x = e.v;
}
}

if (x == -1) return false;

for (int i = 0; i < n; i++) x = parent[x];

cycle.clear();
for (int v = x;; v = parent[v]) {
    cycle.push_back(v);
    if (v == x && cycle.size() > 1) break;
}
reverse(cycle.begin(), cycle.end());
return true;
}

```

7.4 Floyd-Warshall (All-Pairs Shortest Path)

Computes all-pairs shortest paths in $O(V^3)$. Propagates negative cycles.

```

void floyd_warshall(int n, vector<vector<int>>& d) {
    for (int i = 1; i ≤ n; i++) d[i][i] = min(d[i][i], 0LL);
    for (int k = 1; k ≤ n; k++)
        for (int i = 1; i ≤ n; i++)
            for (int j = 1; j ≤ n; j++)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    // Propagate -INF for negative cycles:
    for (int k = 1; k ≤ n; k++)
        if (d[k][k] < 0)
            for (int i = 1; i ≤ n; i++)
                for (int j = 1; j ≤ n; j++)
                    if (d[i][k] < INF && d[k][j] < INF) d[i][j] = -INF;
}

```

7.5a Topological Sort

Kahn's algorithm for a DAG. If the result has fewer than N vertices, the graph contains a cycle.

```

vector<int> topo_sort(int n, const vector<vector<int>>& g) {
    vector<int> indeg(n + 1);

    for (int u = 1; u ≤ n; u++)
        for (int v : g[u]) indeg[v]++;

    queue<int> q;
    for (int i = 1; i ≤ n; i++)
        if (indeg[i] == 0) q.push(i);

    vector<int> order;

    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);

        for (int v : g[u])
            if (--indeg[v] == 0) q.push(v);
    }

    return order; // size < n => cycle exists
}

```

7.5b Kruskal MST

Minimum spanning tree in $O(E \log E)$ using DSU.

```

struct WEdge {
    int u, v;
    long long w;
    bool operator<(const WEdge& other) const {
        return w < other.w;
    }
}

```

```

};

pair<long long, vector<WEde>>
kruskal(int n, vector<WEde> edges) {
    sort(edges.begin(), edges.end());
    DSU dsu(n);

    long long cost = 0;
    vector<WEde> used;

    for (auto e : edges) {
        if (dsu.unite(e.u, e.v)) {
            cost += e.w;
            used.push_back(e);
        }
    }

    return {cost, used}; // used.size() == n-1 iff connected
}

```

7.5c Prim MST

MST for a connected weighted undirected graph in $O(E \log V)$.

```

long long prim_mst(int n,
    const vector<vector<pair<int,int>>>& g,
    int start = 1) {
    vector<int> vis(n + 1, false);
    priority_queue<pair<long long,int>,
        vector<pair<long long,int>>,
        greater<pair<long long,int>>> pq;

    pq.push({0, start});
    long long cost = 0;
    int taken = 0;

    while (!pq.empty()) {
        auto [w, u] = pq.top(); pq.pop();
        if (vis[u]) continue;

        vis[u] = true;
        taken++;
        cost += w;

        for (auto [v, wt] : g[u])
            if (!vis[v])
                pq.push({wt, v});
    }

    return taken == n ? cost : INF;
}

```

7.5d Bipartite Check

Two-coloring in $O(V + E)$.

```

bool is_bipartite(int n, const vector<vector<int>>& g) {
    vector<int> color(n + 1, -1);

    for (int s = 1; s ≤ n; s++) {
        if (color[s] ≠ -1) continue;

        queue<int> q;
        q.push(s);
        color[s] = 0;

        while (!q.empty()) {
            int u = q.front(); q.pop();

            for (int v : g[u]) {
                if (color[v] == -1) {
                    color[v] = color[u] ^ 1;
                    q.push(v);
                } else if (color[v] == color[u]) {
                    return false;
                }
            }
        }
    }
}

```

```

    }
}

return true;
}

```

7.5 Strongly Connected Components (Kosaraju's)

Two DFS passes. Builds SCC ids and the condensation DAG. For very deep graphs, recursive DFS may need an iterative version.

```

struct KosarajuSCC {
    int n, comps = 0;
    vector<vector<int>> g, rg, dag;
    vector<int> order, comp;
    vector<bool> vis;

    KosarajuSCC(int n) : n(n), g(n + 1), rg(n + 1),
        comp(n + 1, -1), vis(n + 1, false) {}

    void add_edge(int u, int v) {
        g[u].push_back(v);
        rg[v].push_back(u);
    }

    void dfs1(int u) {
        vis[u] = true;
        for (int v : g[u])
            if (!vis[v]) dfs1(v);
        order.push_back(u);
    }

    void dfs2(int u, int c) {
        comp[u] = c;
        for (int v : rg[u])
            if (comp[v] == -1) dfs2(v, c);
    }

    void build() {
        order.clear();
        comp.assign(n + 1, -1);
        vis.assign(n + 1, false);
        comps = 0;

        for (int i = 1; i ≤ n; i++)
            if (!vis[i]) dfs1(i);

        reverse(order.begin(), order.end());
        for (int u : order)
            if (comp[u] == -1) dfs2(u, comps++);

        dag.assign(comps, {});
        for (int u = 1; u ≤ n; u++)
            for (int v : g[u])
                if (comp[u] ≠ comp[v])
                    dag[comp[u]].push_back(comp[v]);
    }
};

```

7.6 Strongly Connected Components (Tarjan's)

Single-pass DFS with explicit stack and entry/low values to find SCCs in $O(V + E)$.

```

struct TarjanSCC {
    int n, timer = 0, comps = 0;
    vector<vector<int>> g;
    vector<int> tin, low, comp;
    vector<bool> in_st;
    stack<int> st;

    TarjanSCC(int n) : n(n), g(n + 1), tin(n + 1, -1), low(n + 1, -1),
        comp(n + 1, -1), in_st(n + 1, false) {}

    void add_edge(int u, int v) { g[u].push_back(v); }
}

```

```

void dfs(int u) {
    tin[u] = low[u] = ++timer;
    st.push(u); in_st[u] = true;
    for (int v : g[u]) {
        if (tin[v] == -1) {
            dfs(v);
            low[u] = min(low[u], low[v]);
        } else if (in_st[v]) {
            low[u] = min(low[u], tin[v]);
        }
    }

    if (low[u] == tin[u]) {
        while (true) {
            int v = st.top(); st.pop();
            in_st[v] = false;
            comp[v] = comps;
            if (v == u) break;
        }
        comps++;
    }
}

void build() { for (int i = 1; i ≤ n; i++) if (tin[i] == -1) dfs(i); }
};

```

7.7 2-SAT Solver (via SCC)

Variables are 1-indexed. Node $2i$ means false, node $2i + 1$ means true.

```

struct TwoSAT {
    int n;
    KosarajuSCC scc;

    TwoSAT(int n) : n(n), scc(2 * n + 2) {}

    void add_clause(int u, bool vu, int v, bool vv) {
        int a = 2 * u + (vu ? 1 : 0);
        int b = 2 * v + (vv ? 1 : 0);

        scc.add_edge(a ^ 1, b);
        scc.add_edge(b ^ 1, a);
    }

    bool solve(vector<bool>& assignment) {
        scc.build();
        assignment.assign(n + 1, false);

        for (int i = 1; i ≤ n; i++) {
            if (scc.comp[2 * i] == scc.comp[2 * i + 1])
                return false;

            // Kosaraju here numbers SCCs in source-to-sink order.
            assignment[i] =
                scc.comp[2 * i + 1] > scc.comp[2 * i];
        }

        return true;
    }
};

```

7.8 Articulation Points & Bridges

Undirected graph in $O(V + E)$. Uses edge ids, so parallel edges are handled correctly.

```

struct BridgesAndArt {
    int n, timer = 0, edge_id = 0;
    vector<vector<pair<int,int>>> g; // {to, edge_id}
    vector<int> tin, low;
    vector<bool> is_art;
    vector<pair<int,int>> bridges;

    BridgesAndArt(int n) : n(n), g(n + 1), tin(n + 1, -1),
        low(n + 1, -1), is_art(n + 1, false) {}

    void add_edge(int u, int v) {

```

```

int id = edge_id++;
g[u].push_back({v, id});
g[v].push_back({u, id});
}

void dfs(int u, int pe = -1) {
    tin[u] = low[u] = ++timer;
    int children = 0;

    for (auto [v, id] : g[u]) {
        if (id == pe) continue;

        if (tin[v] != -1) {
            low[u] = min(low[u], tin[v]);
        } else {
            dfs(v, id);
            low[u] = min(low[u], low[v]);

            if (low[v] > tin[u])
                bridges.push_back({u, v});

            if (low[v] ≥ tin[u] && pe != -1)
                is_art[u] = true;

            children++;
        }
    }

    if (pe == -1 && children > 1)
        is_art[u] = true;
}

void build() {
    timer = 0;
    bridges.clear();
    fill(tin.begin(), tin.end(), -1);
    fill(low.begin(), low.end(), -1);
    fill(is_art.begin(), is_art.end(), false);

    for (int i = 1; i ≤ n; i++)
        if (tin[i] == -1) dfs(i);
}

```

7.8a Euler Path / Circuit (Undirected)

Use Hierholzer's algorithm. An Euler circuit exists iff every non-isolated vertex has even degree; an Euler path iff exactly 0 or 2 vertices have odd degree.

```

vector<vector<pair<int,int>>> eg;
vector<int> used_edge, euler;

```

```

void euler_dfs(int u) {
    while (!eg[u].empty()) {
        auto [v, id] = eg[u].back();
        eg[u].pop_back();
        if (used_edge[id]) continue;
        used_edge[id] = 1;
        euler_dfs(v);
    }
    euler.push_back(u);
}

// After running from a suitable start:
// reverse(euler);
// It is valid iff euler.size() == E + 1.

```

7.8b Shortest Path in a DAG

Topological order + one pass gives $O(V+E)$ shortest paths, even with negative edge weights.

```

vector<long long> dag_shortest(int n, int src,
    const vector<vector<pair<int,int>>>& g,
    const vector<int>& topo) {

```

```

    vector<long long> dist(n + 1, INF);
    dist[src] = 0;

    for (int u : topo) {
        if (dist[u] == INF) continue;
        for (auto [v, w] : g[u])
            dist[v] = min(dist[v], dist[u] + w);
    }
    return dist;
}

```

7.9 Tree Flattening (Euler Tour)

Maps tree subtrees into contiguous 1D array segments $[tin[u], tout[u]]$ for range data structures.

```

vector<int> tin, tout;
int timer = 0;
void euler_tour(int u, int p, const vector<vector<int>>& g) {
    tin[u] = ++timer;
    for (int v : g[u]) if (v != p) euler_tour(v, u, g);
    tout[u] = timer;
}

```

7.10 Binary Lifting & LCA

Precomputation $O(N \log N)$; LCA and k -th ancestor in $O(\log N)$. Recursive DFS may need an iterative version on very deep trees.

```

struct LCA {
    int n, LG;
    vector<vector<int>> up;
    vector<int> depth;

    LCA(int n, const vector<vector<int>>& g, int root = 1)
        : n(n), LG(__lg(max(1LL, n)) + 1),
          up(LG, vector<int>(n + 1)), depth(n + 1) {

        auto dfs = [&](auto self, int u, int p, int d) → void {
            up[0][u] = p;
            depth[u] = d;

            for (int j = 1; j < LG; j++)
                up[j][u] = up[j - 1][up[j - 1][u]];

            for (int v : g[u])
                if (v != p) self(self, v, u, d + 1);
        };

        dfs(dfs, root, 0, 0);
    }

    int kth_ancestor(int u, int k) const {
        for (int j = 0; j < LG; j++)
            if (k & (1LL << j))
                u = up[j][u];
        return u;
    }

    int get_lca(int u, int v) const {
        if (depth[u] < depth[v]) swap(u, v);

        int diff = depth[u] - depth[v];
        for (int j = LG - 1; j ≥ 0; j--)
            if (diff & (1LL << j))
                u = up[j][u];

        if (u == v) return u;

        for (int j = LG - 1; j ≥ 0; j--) {
            if (up[j][u] != up[j][v]) {
                u = up[j][u];
                v = up[j][v];
            }
        }
    }
}

```

```

    return up[0][u];
}
};

```

7.11 Heavy-Light Decomposition (HLD)

Decomposes tree into $O(\log N)$ heavy paths. Supports path queries and updates via segment tree.

```

struct HLD {
    int n, cur_pos = 1;
    vector<int> parent, depth, heavy, head, pos, sz;
    vector<vector<int>> g;
    HLD(int n) : n(n), parent(n + 1), depth(n + 1), heavy(n + 1, -1),
        head(n + 1), pos(n + 1), sz(n + 1), g(n + 1) {}
    void add_edge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }

    int dfs(int u, int p, int d) {
        parent[u] = p; depth[u] = d; sz[u] = 1;
        int max_c = 0;
        for (int v : g[u]) {
            if (v == p) continue;
            int c_sz = dfs(v, u, d + 1);
            sz[u] += c_sz;
            if (c_sz > max_c) { max_c = c_sz; heavy[u] = v; }
        }
        return sz[u];
    }

    void decompose(int u, int h) {
        head[u] = h; pos[u] = cur_pos++;
        if (heavy[u] != -1) decompose(heavy[u], h);
        for (int v : g[u]) {
            if (v != parent[u] && v != heavy[u]) decompose(v, v);
        }
    }

    void init(int root = 1) { dfs(root, 0, 0); decompose(root, root); }
};

```

7.12 DSU on Tree (Sack / Small-to-Large on Trees)

Solves offline subtree queries in $O(N \log N)$ by keeping the heavy child's contribution.

```

int sz_tree[N], heavy_child[N];
void dfs_sz(int u, int p, const vector<vector<int>>& g) {
    sz_tree[u] = 1; heavy_child[u] = -1; int max_c = 0;
    for (int v : g[u]) if (v != p) {
        dfs_sz(v, u, g);
        sz_tree[u] += sz_tree[v];
        if (sz_tree[v] > max_c) { max_c = sz_tree[v]; heavy_child[u] = v; }
    }

    void dfs_sack(int u, int p, bool keep, const vector<vector<int>>& g) {
        for (int v : g[u]) if (v != p && v != heavy_child[u]) dfs_sack(v, u, false, g);
        if (heavy_child[u] != -1) dfs_sack(heavy_child[u], u, true, g);
        // Add light children subtrees and node u to DS
        // Answer queries for u
        if (!keep) { /* Remove all elements in subtree u from DS */ }
    }
}

```

7.13 Dinic's Algorithm (Maximum Flow)

Maximum flow with BFS level graph and DFS blocking flow. Practical and standard for many contest constraints.

```

struct Dinic {
    struct Edge {
        int to, rev;
        long long cap;
    };

    int n, s, t;

```

```

vector<vector<Edge>> g;
vector<int> level, ptr;

Dinic(int n, int s, int t)
    : n(n), s(s), t(t), g(n + 1),
      level(n + 1), ptr(n + 1) {}

void add_edge(int u, int v, long long cap) {
    Edge a(v, (int)g[v].size(), cap);
    Edge b(u, (int)g[u].size(), 0);
    g[u].push_back(a);
    g[v].push_back(b);
}

bool bfs() {
    fill(level.begin(), level.end(), -1);
    queue<int> q;
    q.push(s);
    level[s] = 0;

    while (!q.empty()) {
        int u = q.front(); q.pop();

        for (auto &e : g[u]) {
            if (e.cap > 0 || level[e.to] != -1) continue;
            level[e.to] = level[u] + 1;
            q.push(e.to);
        }
    }
    return level[t] != -1;
}

long long dfs(int u, long long pushed) {
    if (!pushed) return 0;
    if (u == t) return pushed;

    for (int &cid = ptr[u]; cid < (int)g[u].size(); cid++) {
        Edge &e = g[u][cid];
        if (level[e.to] != level[u] + 1 || e.cap <= 0) continue;

        long long tr = dfs(e.to, min(pushed, e.cap));
        if (!tr) continue;

        e.cap -= tr;
        g[e.to][e.rev].cap += tr;
        return tr;
    }
    return 0;
}

long long max_flow() {
    long long flow = 0;
    while (bfs()) {
        fill(ptr.begin(), ptr.end(), 0);
        while (long long pushed = dfs(s, INF))
            flow += pushed;
    }
    return flow;
}
};

```

7.14 Maximum Bipartite Matching (Kuhn's)

Simple $O(V \cdot E)$ augmenting path algorithm for bipartite matching.

```

bool dfs_kuhn(int u, const vector<vector<int>>& g, vector<int>&
    match, vector<bool>& vis) {
    if (vis[u]) return false;
    vis[u] = true;
    for (int v : g[u]) {
        if (match[v] == -1 || dfs_kuhn(match[v], g, match, vis)) {
            match[v] = u; return true;
        }
    }
}

```

```

}
return false;
}

int max_bipartite_matching(int n_left, int n_right, const vector<
    vector<int>>& g) {
    vector<int> match(n_right + 1, -1);
    int res = 0;
    for (int u = 1; u <= n_left; u++) {
        vector<bool> vis(n_left + 1, false);
        if (dfs_kuhn(u, g, match, vis)) res++;
    }
    return res;
}

```

7.15 Tree Diameter (2-BFS)

BFS from node 1 gives farthest node u; BFS from u gives farthest node v and diameter.

```

pair<int, int> bfs_farthest(int start, int n, const vector<vector<
    int>>& g) {
    vector<int> dist(n + 1, -1);
    queue<int> q; q.push(start); dist[start] = 0;
    int far_node = start;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (dist[u] > dist[far_node]) far_node = u;
        for (int v : g[u]) if (dist[v] == -1) { dist[v] = dist[u] + 1; q
            .push(v); }
    }
    return {far_node, dist[far_node]};
}

int tree_diameter(int n, const vector<vector<int>>& g) {
    int u = bfs_farthest(1, n, g).first;
    return bfs_farthest(u, n, g).second;
}

```

8. Dynamic Programming

8.1 Classical DP (Knapsack, Coins, LIS, LCS, Edit Dist)

Essential competitive programming dynamic programming templates.

```

// 0/1 Knapsack (Reverse iteration):
for (int i = 0; i < n; i++)
    for (int j = W; j >= wt[i]; j--) dp[j] = max(dp[j], dp[j - wt[i]]
        + val[i]);

// Unbounded Knapsack (Forward iteration):
for (int i = 0; i < n; i++)
    for (int j = wt[i]; j <= W; j++) dp[j] = max(dp[j], dp[j - wt[i]]
        + val[i]);

```

```

// LIS in  $O(N \log N)$ :
vector<int> tails;
for (int x : a) {
    auto it = lower_bound(tails.begin(), tails.end(), x);
    if (it == tails.end()) tails.push_back(x);
    else *it = x;
}
int lis_len = tails.size();

```

```

// LCS in  $O(N \cdot M)$ :
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        dp[i][j] = (s[i-1] == t[j-1] ? dp[i-1][j-1] + 1 : max(dp[i-1][j]
            , dp[i][j-1]));

// Edit Distance:
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++) {
        if (s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1];
        else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
    }

```

8.2 Kadane's Algorithm & Max Submatrix Sum

Max subarray sum in $O(N)$; 2D submatrix max sum in $O(N^2M)$ by collapsing columns.

```

int kadane(const vector<int>& a) {
    int cur = 0, ans = -INF;
    for (int x : a) { cur = max(x, cur + x); ans = max(ans, cur); }
    return ans;
}

int max_submatrix_sum(int n, int m, const vector<vector<int>>& mat)
    {
        int best = -INF;
        for (int top = 0; top < n; top++) {
            vector<int> col_sum(m, 0);
            for (int bot = top; bot < n; bot++) {
                for (int j = 0; j < m; j++) col_sum[j] += mat[bot][j];
                best = max(best, kadane(col_sum));
            }
        }
        return best;
    }
}

```

8.3 Subset Sum via Bitset (Binary Splitting)

Bounded subset sum. Replace each multiplicity with 1, 2, 4, ... sized groups. Bitset shifts make each group fast.

```

bitset<1000005> bounded_subset_sum(const vector<pair<int, int>>&
    items) {
    bitset<1000005> bs;
    bs[0] = 1;

    for (auto [val, cnt] : items) {
        for (int take = 1; cnt > 0; take <= 1) {
            int k = min(take, cnt);
            bs |= bs << (val * k);
            cnt -= k;
        }
    }
    return bs;
}

```

8.4 Interval DP (Matrix Chain Multiplication)

DP over range lengths: $dp[l][r] = \min_{l \leq k < r} (dp[l][k] + dp[k+1][r] + cost)$.

```

for (int len = 2; len <= n; len++) {
    for (int l = 1; l + len - 1 <= n; l++) {
        int r = l + len - 1;
        dp[l][r] = INF;
        for (int k = l; k < r; k++) {
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k+1][r] + cost(l, k,
                r));
        }
    }
}

```

8.5 Digit DP Template

State dimensions depend on the problem. This example counts numbers in $[0, N]$ by digit sum; adjust MAXSTATE.

```

const int MAXSTATE = 200;
long long dp[20][2][MAXSTATE];
string num_str;

```

```

long long digit_dp(int pos, bool tight, bool lead, int sum) {
    if (pos == (int)num_str.size())
        return 1;
}

```

```

long long &memo = dp[pos][tight][lead][sum];
if (memo != -1) return memo;

```

```

int limit = tight ? num_str[pos] - '0' : 9;
long long ans = 0;

```

```

for (int dig = 0; dig <= limit; dig++) {

```

```

    bool ntight = tight && (dig == num_str[pos] - '0');
    bool nlead = lead && (dig == 0);
    int nsum = nlead ? 0 : sum + dig;
    if (nsum < MAXSTATE)
        ans += digit_dp(pos + 1, ntight, nlead, nsum);
}

return memo = ans;
}

long long count_up_to(long long n) {
    if (n < 0) return 0;
    num_str = to_string(n);
    memset(dp, -1, sizeof(dp));
    return digit_dp(0, true, true, 0);
}

```

8.6 Bitmask DP (TSP / Shortest Hamiltonian Cycle)

Exact TSP in $O(N^2 2^N)$ time and $O(N 2^N)$ memory. Usually practical only around $N \leq 20$. Memory is large.

```

long long tsp(int n, const vector<vector<long long>>& dist) {
    const long long INF2 = (1LL << 60);
    int S = 1 << n;

    vector<vector<long long>> dp(S, vector<long long>(n, INF2));
    dp[1][0] = 0;

    for (int mask = 1; mask < S; mask++) {
        for (int u = 0; u < n; u++) {
            if (dp[mask][u] == INF2) continue;

            for (int v = 0; v < n; v++) {
                if (mask & (1 << v)) continue;
                int nxt = mask | (1 << v);
                dp[nxt][v] =
                    min(dp[nxt][v], dp[mask][u] + dist[u][v]);
            }
        }

        long long ans = INF2;
        for (int u = 1; u < n; u++)
            ans = min(ans, dp[S - 1][u] + dist[u][0]);

    }

    return ans;
}
// For n=20 this is roughly 160 MB with long long, plus overhead.

```

8.7 Tree DP (Maximum Weight Independent Set)

Chooses a subset of nodes with no two adjacent so total weight is maximized.

```

int dp[N][2]; // dp[u][0]: u not picked; dp[u][1]: u picked
void dfs_tree_dp(int u, int p, const vector<vector<int>>& g, const
    vector<int>& weight) {
    dp[u][0] = 0;
    dp[u][1] = weight[u];
    for (int v : g[u]) {
        if (v == p) continue;
        dfs_tree_dp(v, u, g, weight);
        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
}

```

8.7a Rerooting DP: Sum of Distances

Classic example on a tree. First compute the answer for one root, then reroot in $O(N)$.

```

vector<int> sub;
vector<long long> dp_tree, ans_tree;

void dfs1(int u, int p, const vector<vector<int>>& g) {

```

```

    sub[u] = 1;
    dp_tree[u] = 0;

    for (int v : g[u]) {
        if (v == p) continue;
        dfs1(v, u, g);
        sub[u] += sub[v];
        dp_tree[u] += dp_tree[v] + sub[v];
    }

    void dfs2(int u, int p, int n,
        const vector<vector<int>>& g) {
        for (int v : g[u]) {
            if (v == p) continue;

            ans_tree[v] =
                ans_tree[u] - sub[v] + (n - sub[v]);

            dfs2(v, u, n, g);
        }
    }

    // Initialize:
    // sub.resize(n+1); dp_tree.resize(n+1); ans_tree.resize(n+1);
    // dfs1(1,0,g); ans_tree[1]=dp_tree[1]; dfs2(1,0,n,g);

    8.8 SOS DP (Sum Over Subsets / Fast Zeta Transform)
    Computes  $F(mask) = \sum_{sub \subseteq mask} A(sub)$  in  $O(B \cdot 2^B)$  time.

    void sos_dp(vector<int>& f, int bits) {
        for (int b = 0; b < bits; b++) {
            for (int mask = 0; mask < (1 << bits); mask++) {
                if (mask & (1 << b)) f[mask] += f[mask ^ (1 << b)];
            }
        }

        // Inverse Zeta Transform (Möbius transform on bitmasks):
        void inverse_sos_dp(vector<int>& f, int bits) {
            for (int b = 0; b < bits; b++) {
                for (int mask = 0; mask < (1 << bits); mask++) {
                    if (mask & (1 << b)) f[mask] -= f[mask ^ (1 << b)];
                }
            }
        }
    }

```

8.9 Knuth's DP Optimization

Reduces interval DP from $O(N^3)$ to $O(N^2)$ only when the required quadrangle/monotonicity conditions hold. This is a framework, not a universal optimization.

```

// dp[i][j] = min over k in [opt[i][j-1], opt[i+1][j]-like range]
// with opt[i][j-1] ≤ opt[i][j] ≤ opt[i+1][j].

for (int i = 1; i ≤ n; i++)
    opt[i][i] = i, dp[i][i] = 0;

for (int len = 2; len ≤ n; len++) {
    for (int l = 1; l + len - 1 ≤ n; l++) {
        int r = l + len - 1;
        dp[l][r] = INF;

        int L = opt[l][r - 1];
        int R = min(r - 1, opt[l + 1][r]);

        for (int k = L; k ≤ R; k++) {
            long long cand =
                dp[l][k] + dp[k + 1][r] + C(l, r);

            if (cand < dp[l][r]) {
                dp[l][r] = cand;
                opt[l][r] = k;
            }
        }
    }
}

```

```

    }
    }
}

8.10 Li Chao Tree (Minimum Line Queries)
Maintains lines  $y = mx + b$  and answers minimum values on an integer  $x$ 
domain in  $O(\log C)$  per operation.

struct Line {
    long long m = 0, b = INF;

    long long eval(long long x) const {
        return (long long)((__int128)m * x + b);
    }
};

```

```

struct LiChaoTree {
    struct Node {
        Line line;
        int lc = -1, rc = -1;
    };

    vector<Node> tr;
    long long X_L, X_R;

    LiChaoTree(long long L, long long R)
        : X_L(L), X_R(R) {
        tr.push_back({Line(), -1, -1});
    }

    void add_line(Line nw, int node, long long l, long long r) {
        long long mid = l + (r - l) / 2;

        bool lef = nw.eval(l) < tr[node].line.eval(l);
        bool mid_better = nw.eval(mid) < tr[node].line.eval(mid);

        if (mid_better) swap(nw, tr[node].line);
        if (l == r) return;

        if (lef ≠ mid_better) {
            if (tr[node].lc == -1) {
                tr[node].lc = tr.size();
                tr.push_back({Line(), -1, -1});
            }
            add_line(nw, tr[node].lc, l, mid);
        } else {
            if (tr[node].rc == -1) {
                tr[node].rc = tr.size();
                tr.push_back({Line(), -1, -1});
            }
            add_line(nw, tr[node].rc, mid + 1, r);
        }
    }

    void add_line(long long m, long long b) {
        add_line({m, b}, 0, X_L, X_R);
    }

    long long query(long long x, int node,
        long long l, long long r) const {
        if (node == -1) return INF;

        long long ans = tr[node].line.eval(x);
        if (l == r) return ans;

        long long mid = l + (r - l) / 2;
        if (x ≤ mid)
            return min(ans, query(x, tr[node].lc, l, mid));
        return min(ans, query(x, tr[node].rc, mid + 1, r));
    }
}

```

```
long long query(long long x) const {
    return query(x, 0, X_L, X_R);
}
};
```

9. Greedy, Searching & Techniques

9.1 Binary Search (First True & Last True)

Standard monotonic binary search implementations preventing infinite loops.

```
// First True (Smallest x where check(x) == true):
int low = 0, high = 1e18, ans = -1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (check(mid)) { ans = mid; high = mid - 1; }
    else low = mid + 1;
}
```

```
// Last True (Largest x where check(x) == true):
low = 0, high = 1e18, ans = -1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (check(mid)) { ans = mid; low = mid + 1; }
    else high = mid - 1;
}
```

9.2 Floating Point Binary Search

Use a fixed number of iterations. The final absolute error is roughly the initial interval width divided by 2^{iterations}.

```
double lo = 0.0, hi = 1e9;
for (int it = 0; it < 100; it++) {
    double mid = (lo + hi) / 2.0;
    if (check(mid)) hi = mid;
    else lo = mid;
}
```

9.3 Ternary Search (Unimodal Functions)

Unimodal search for a minimum or maximum.

```
// Continuous Ternary Search (Finding minimum):
double l = -1e9, r = 1e9;
for (int i = 0; i < 200; i++) {
    double m1 = l + (r - l) / 3.0;
    double m2 = r - (r - l) / 3.0;
    if (f(m1) > f(m2)) l = m1;
    else r = m2;
}
```

```
// Discrete Ternary Search (Integers):
int low = 1, high = 1e9;
while (high - low > 2) {
    int m1 = low + (high - low) / 3;
    int m2 = high - (high - low) / 3;
    if (f(m1) < f(m2)) high = m2;
    else low = m1;
}
int ans = f(low);
for (int i = low + 1; i <= high; i++) ans = min(ans, f(i));
```

9.4 Two Pointers Technique

Maintains linear relationships across two indices moving in the same or opposite directions.

```
// Two Sum in Sorted Array:
int l = 0, r = n - 1;
while (l < r) {
    int sum = a[l] + a[r];
    if (sum == target) { /* Found */ break; }
    else if (sum < target) l++;
    else r--;
}
```

9.5 Sliding Window (AtMost Trick)

```
Solves exact count queries: Count(K) = AtMost(K) - AtMost(K - 1).
// Number of subarrays with at most K distinct elements:
int at_most_k_distinct(const vector<int>& a, int k) {
    if (k < 0) return 0;
    unordered_map<int, int> freq;
    int l = 0, ans = 0;
    for (int r = 0; r < (int)a.size(); r++) {
        if (freq[a[r]]++ == 0) k--;
        while (k < 0) {
            if (--freq[a[l]] == 0) k++;
            l++;
        }
        ans += (r - l + 1);
    }
    return ans;
}
int exact_k_distinct(const vector<int>& a, int k) {
    return at_most_k_distinct(a, k) - at_most_k_distinct(a, k - 1);
}
```

9.6 Meet in the Middle

Splits N ≤ 40 search spaces into two halves of size N/2, then combines via sorting/binary search.

```
void generate_subset_sums(const vector<int>& a, vector<int>& sums) {
    int n = a.size();
    for (int mask = 0; mask < (1 << n); mask++) {
        int cur = 0;
        for (int i = 0; i < n; i++) if (mask & (1 << i)) cur += a[i];
        sums.push_back(cur);
    }
    sort(sums.begin(), sums.end());
}
int meet_in_the_middle(const vector<int>& a, int target) {
    int n = a.size();
    vector<int> left(a.begin(), a.begin() + n / 2);
    vector<int> right(a.begin() + n / 2, a.end());
    vector<int> sumL, sumR;
    generate_subset_sums(left, sumL);
    generate_subset_sums(right, sumR);
    int best = 0;
    for (int s : sumL) {
        if (s <= target) {
            auto it = upper_bound(sumR.begin(), sumR.end(), target - s);
            if (it != sumR.begin()) best = max(best, s + *(--it));
        }
    }
    return best;
}
```

10. Computational Geometry

10.1 Point Structure & Vector Fundamentals

Floating-point geometry helper. Do not use EPS inside a sorting comparator. For exact integer coordinates, prefer the short integer variant below.

```
const double EPS = 1e-9;
const double PI = acos(-1.0);

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}

    Point operator+(const Point& p) const { return {x + p.x, y + p.y}; }
    Point operator-(const Point& p) const { return {x - p.x, y - p.y}; }
    Point operator*(double k) const { return {x * k, y * k}; }
    Point operator/(double k) const { return {x / k, y / k}; }

    bool operator<(const Point& p) const {
        if (x != p.x) return x < p.x;

```

```
        return y < p.y;
    }
};

double dot(Point a, Point b) {
    return a.x * b.x + a.y * b.y;
}
double cross(Point a, Point b) {
    return a.x * b.y - a.y * b.x;
}
double dist_sq(Point a, Point b) {
    double dx = a.x - b.x, dy = a.y - b.y;
    return dx * dx + dy * dy;
}
double dist(Point a, Point b) {
    return hypot(a.x - b.x, a.y - b.y);
}

int orient(Point a, Point b, Point c) {
    double v = cross(b - a, c - a);
    if (v > EPS) return 1;
    if (v < -EPS) return -1;
    return 0;
}
```

```
Point rotate_point(Point p, double ang) {
    double c = cos(ang), s = sin(ang);
    return {p.x * c - p.y * s, p.x * s + p.y * c};
}
```

```
// Exact integer orientation:
struct PointLL {
    long long x, y;

    bool operator<(const PointLL& p) const {
        return x != p.x ? x < p.x : y < p.y;
    }
};
```

```
__int128 cross(PointLL a, PointLL b, PointLL c) {
    return (__int128)(b.x - a.x) * (c.y - a.y)
        - (__int128)(b.y - a.y) * (c.x - a.x);
}
```

10.2 Lines, Segments & Intersections

Segment intersection and point-to-segment distance.

```
bool on_segment(Point p, Point a, Point b) {
    return orient(a, b, p) == 0 &&
        min(a.x, b.x) <= p.x <= max(a.x, b.x) + EPS &&
        min(a.y, b.y) <= p.y <= max(a.y, b.y) + EPS;
}
bool intersect_segments(Point a, Point b, Point c, Point d) {
    int o1 = orient(a, b, c), o2 = orient(a, b, d);
    int o3 = orient(c, d, a), o4 = orient(c, d, b);
    if (o1 != o2 && o3 != o4) return true;
    if (on_segment(c, a, b) || on_segment(d, a, b)) return true;
    if (on_segment(a, c, d) || on_segment(b, c, d)) return true;
    return false;
}
```

```
Point line_intersection(Point a, Point b, Point c, Point d) {
    double cp = cross(b - a, d - c);
    return a + (b - a) * (cross(c - a, d - c) / cp);
}
double dist_to_segment(Point p, Point a, Point b) {
    if (dot(b - a, p - a) <= 0) return dist(p, a);
    if (dot(a - b, p - b) <= 0) return dist(p, b);
    return abs(cross(b - a, p - a)) / dist(a, b);
}
```

10.3 Polygon Area (Shoelace Formula)

Vertices ordered around the polygon. Use squared/doubled area when coordi-

notes are integer and exactness matters.

```
double polygon_area(const vector<Point>& p) {
    long long n = p.size();
    double area = 0;

    for (int i = 0; i < n; i++)
        area += cross(p[i], p[(i + 1) % n]);

    return abs(area) / 2.0;
}

__int128 polygon_area2(const vector<PointLL>& p) {
    int n = p.size();
    __int128 twice = 0;

    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        twice += (__int128)p[i].x * p[j].y
            - (__int128)p[j].x * p[i].y;
    }

    return twice ≥ 0 ? twice : -twice;
}
```

10.4 Convex Hull (Monotonic Chain / Andrew's)

$O(N \log N)$. This version removes duplicate points and interior collinear points.

```
vector<Point> convex_hull(vector<Point> pts) {
    sort(pts.begin(), pts.end());
    pts.erase(unique(pts.begin(), pts.end(), [](const Point& a, const
        Point& b) {
            return a.x == b.x && a.y == b.y;
        }), pts.end());

    int n = pts.size();
    if (n ≤ 2) return pts;

    vector<Point> h(2 * n);
    int k = 0;

    for (auto p : pts) {
        while (k ≥ 2 &&
            cross(h[k - 1] - h[k - 2], p - h[k - 2]) ≤ EPS)
            k--;
        h[k++] = p;
    }

    for (int i = n - 2, t = k + 1; i ≥ 0; i--) {
        while (k ≥ t &&
            cross(h[k - 1] - h[k - 2], pts[i] - h[k - 2]) ≤ EPS)
            k--;
        h[k++] = pts[i];
    }

    h.resize(k - 1);
    return h;
}
```

10.5 Rotating Calipers (Polygon Diameter)

Maximum distance among vertices of a convex polygon in $O(N)$. Compare squared distances to avoid repeated square roots.

```
double rotating_calipers_diameter(const vector<Point>& h) {
    int n = h.size();
    if (n ≤ 1) return 0;
    if (n == 2) return dist(h[0], h[1]);

    double best = 0;
    int j = 1;

    for (int i = 0; i < n; i++) {
        int ni = (i + 1) % n;
```

```
        while (true) {
            int nj = (j + 1) % n;
            double cur = abs(cross(h[ni] - h[i], h[j] - h[i]));
            double nxt = abs(cross(h[ni] - h[i], h[nj] - h[i]));
            if (nxt > cur + EPS) j = nj;
            else break;
        }

        best = max(best, dist_sq(h[i], h[j]));
        best = max(best, dist_sq(h[ni], h[j]));
    }

    return sqrt(best);
}
```

10.6 Point in Polygon (Ray Casting)

Returns 0 outside, 1 strictly inside, 2 on the boundary. Works for a simple polygon.

```
bool on_segment(Point p, Point a, Point b) {
    return orient(a, b, p) == 0 &&
        min(a.x, b.x) ≤ p.x + EPS &&
        p.x ≤ max(a.x, b.x) + EPS &&
        min(a.y, b.y) ≤ p.y + EPS &&
        p.y ≤ max(a.y, b.y) + EPS;
}
```

```
int point_in_polygon(const vector<Point>& p, Point q) {
    int n = p.size();
    bool inside = false;

    for (int i = 0; i < n; i++) {
        Point a = p[i], b = p[(i + 1) % n];

        if (on_segment(q, a, b)) return 2;

        if ((a.y > q.y) ≠ (b.y > q.y)) {
            double x_cross =
                a.x + (b.x - a.x) * (q.y - a.y) / (b.y - a.y);
            if (q.x < x_cross) inside = !inside;
        }
    }

    return inside ? 1 : 0;
}
```

10.7 Minimum Enclosing Circle

Randomized nested-loop implementation. Expected linear behavior in practice; uses a robust 2-point/3-point boundary construction.

```
struct Circle {
    Point c;
    double r;
};

bool inside(Circle c, Point p) {
    return dist_sq(c.c, p) ≤ (c.r + EPS) * (c.r + EPS);
}
```

```
Circle circle2(Point a, Point b) {
    return {(a + b) / 2.0, dist(a, b) / 2.0};
}
```

```
Circle circle3(Point a, Point b, Point c) {
    double d = 2.0 * cross(b - a, c - a);
```

```
    if (abs(d) < EPS) {
        Circle ab = circle2(a, b);
        Circle ac = circle2(a, c);
        Circle bc = circle2(b, c);

        Circle best = ab;
```

```
        if (inside(best, c) == false) best = ac;
        if (inside(best, b) == false || inside(best, c) == false) best =
            bc;
        return best;
    }

    Point u = b - a, v = c - a;
    double uu = dot(u, u), vv = dot(v, v);

    Point o(
        a.x + (uu * v.y - vv * u.y) / d,
        a.y + (vv * u.x - uu * v.x) / d
    );

    return {o, dist(o, a)};
}
```

```
Point u = b - a, v = c - a;
double uu = dot(u, u), vv = dot(v, v);
```

```
Point o(
    a.x + (uu * v.y - vv * u.y) / d,
    a.y + (vv * u.x - uu * v.x) / d
);
```

```
return {o, dist(o, a)};
}
```

```
Circle min_enclosing_circle(vector<Point> p) {
    static mt19937 rng(chrono::steady_clock::now().time_since_epoch().
        count());
    shuffle(p.begin(), p.end(), rng);

    Circle c({0, 0}, -1);

    for (int i = 0; i < (int)p.size(); i++) {
        if (c.r ≥ 0 && inside(c, p[i])) continue;
```

```
        Circle c({0, 0}, -1);
```

```
        for (int i = 0; i < (int)p.size(); i++) {
            if (c.r ≥ 0 && inside(c, p[i])) continue;
```

```
            c = {p[i], 0};
```

```
            for (int j = 0; j < i; j++) {
                if (inside(c, p[j])) continue;
```

```
                c = circle2(p[i], p[j]);
```

```
                for (int k = 0; k < j; k++) {
                    if (inside(c, p[k])) continue;
                    c = circle3(p[i], p[j], p[k]);
                }
            }
        }
    }
    return c;
}
```

10.8 Classical Geometry Theorems & Formulas

Pick's theorem, cyclic quadrilaterals, incircle, and circumcircle.

- Pick's Theorem: $Area = I + \frac{B}{2} - 1$ (I : interior lattice points, B : boundary points)
- Heron's Formula: $Area = \sqrt{s(s-a)(s-b)(s-c)}$ where $s = \frac{a+b+c}{2}$
- Triangle Inradius & Circumradius: $r = \frac{Area}{s}$, $R = \frac{a \cdot b \cdot c}{4 \cdot Area}$
- Brahmagupta (cyclic): $K^2 = (s-a)(s-b)(s-c)(s-d)$, $s = \frac{a+b+c+d}{2}$
- Fermat Point: For all angles $< 120^\circ$, the three segments to the vertices meet pairwise at 120° .
- Primitive Pythagorean triple: $a = 2st$, $b = s^2 - t^2$, $c = s^2 + t^2$, with $\gcd(s, t) = 1$ and opposite parity.

11. Bit Hacks, Game Theory & STL Reference

11.1 Bit Manipulation Cheatsheet

Common bit hacks and GCC builtins for fast binary arithmetic.

```
// Basic Bit Hacks:
x & -x // Lowest set bit (LSB) value
```

```
x & (x - 1)           // Clears the lowest set bit
x | (x - 1)           // Sets all trailing zero bits to 1
~x & (x + 1)          // Isolates lowest 0-bit
x && !(x & (x - 1))    // Checks if x is a power of 2
(x >> i) & 1           // Gets the i-th bit (0-indexed)
x |= (1LL << i)        // Sets the i-th bit
x &= ~(1LL << i)       // Clears the i-th bit
x ^= (1LL << i)        // Toggles the i-th bit
(1LL << k) - 1         // Mask of k lowest bits
```

```
// GCC Builtins (Append 'll' for 64-bit integers):
__builtin_popcountll(x) // Number of 1-bits
__builtin_clzll(x)      // Leading zeros (Undefined for x = 0!)
__builtin_ctzll(x)      // Trailing zeros (Undefined for x = 0!)
__builtin_parityll(x)    // Parity of set bits (popcount % 2)
63 - __builtin_clzll(x) // MSB position (0-indexed)
```

```
// Iterate all submasks of mask m (excluding 0):
for (int s = m; s > 0; s = (s - 1) & m) { /* process s */ }
```

```
// Next number with the same popcount (GOSPER'S HACK):
int next_popcount(int x) {
    int c = x & -x, r = x + c;
    return r | (((r ^ x) >> 2) / c);
}
```

11.2 Range XOR Sum in $O(1)$

Computes $\bigoplus_{i=0}^n i$ and $\bigoplus_{i=1}^r i$ in $O(1)$.

```
int xor_0_to_n(int n) {
    if (n % 4 == 0) return n;
    if (n % 4 == 1) return 1;
    if (n % 4 == 2) return n + 1;
    return 0;
}
int range_xor(int l, int r) {
    return xor_0_to_n(r) ^ xor_0_to_n(l - 1);
}
```

11.3 Randomized / Zobrist Hashing

Useful probabilistic fingerprint. Equal hashes only mean “probably equal”. Plain XOR is not a safe generic tree-isomorphism multiset hash because duplicate child hashes cancel.

```
mt19937_64 rng_hash(
    chrono::steady_clock::now().time_since_epoch().count());

uint64_t random_val() {
    return rng_hash();
}
```

```
// For a known value range, assign one random value per value and
// XOR prefix contributions. This detects multiset equality
// probabilistically.
vector<uint64_t> rnd(MAXV + 1);
```

11.4 Game Theory & Nim Rules

Impartial games under the normal play convention (last player to move wins).

- Standard Nim: Pile sizes $a_1, a_2, \dots, a_n$. First player wins iff XOR sum $X = a_1 \oplus a_2 \oplus \dots \oplus a_n \neq 0$.
- Subtraction Game: Can remove at most k stones from a pile. Equivalent to Nim with pile sizes $a_i \bmod (k + 1)$.
- Index- k Nim: Can remove from at most k piles in a single turn. Write pile sizes in binary; sum each bit column modulo $k + 1$. First player wins iff at least one bit column sum $\not\equiv 0 \pmod{k + 1}$.

- Misère Nim: Normal play, but the last player to move LOSES.

- If all piles have size 1: First player wins iff total number of piles is EVEN.
- If at least one pile has size > 1 : Play identically to Standard Nim ($X \neq 0 \implies$ First player wins).

- Wythoff’s Game: Two piles of size (n, m) . Can take any from one, or equal from both. Losing states are $(\lfloor k\phi \rfloor, \lfloor k\phi^2 \rfloor)$ where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.6180339887$.

11.5 Sprague-Grundy Theorem

For an impartial game, $g(u) = \text{mex}\{g(v) : u \rightarrow v\}$. Initialize memoization to -1 before use.

```
vector<int> memo_g;

int grundy(int state, const vector<int>& moves) {
    if (state == 0) return 0;

    int &res = memo_g[state];
    if (res != -1) return res;

    vector<char> seen(moves.size() + 64, false);

    for (int mv : moves) {
        if (state < mv) continue;

        int g = grundy(state - mv, moves);
        if (g < (int)seen.size()) seen[g] = true;
    }

    int mex = 0;
    while (mex < (int)seen.size() && seen[mex]) mex++;

    return res = mex;
}
```

```
// Before use:
// memo_g.assign(MAX_STATE + 1, -1);
```

11.6 STL Container Iteration & Erase

Safely deleting elements while iterating across `std::map` and `std::set` without invalidating iterators.

```
for (auto it = s.begin(); it != s.end(); ) {
    if (condition(*it)) it = s.erase(it); // Erase returns next valid iterator
    else ++it;
}
```

11.7 Direction Arrays (Grid Problems)

Standard offset deltas for 4-directional, 8-directional, and Knight moves on 2D grids.

```
// 4-Directional:
const int dx4[4] = {-1, 0, 1, 0};
const int dy4[4] = {0, 1, 0, -1};
```

```
// 8-Directional:
const int dx8[8] = {-1, -1, -1, 0, 0, 1, 1, 1};
const int dy8[8] = {-1, 0, 1, -1, 1, -1, 0, 1};
```

```
// Knight’s Moves:
const int kx[8] = {-2, -2, -1, -1, 1, 1, 2, 2};
const int ky[8] = {-1, 1, -2, 2, -2, 2, -1, 1};
```

11.8 "When to Use What" Contest Cheat Sheet

Fast pattern recognition under contest pressure.

Problem Pattern	Algorithm / DS
Range sum + point update	Fenwick / Segment Tree
Range add + range sum	Lazy Segment Tree
Static RMQ	Sparse Table
Order statistics in a set	PBDS ordered set
XOR pair / maximum XOR	Binary Trie
XOR linear combinations	XOR Basis
Offline range queries	Mo’s Algorithm
Subtree queries	Euler Tour + BIT/SegTree
Tree path queries	HLD + SegTree
LCA / ancestor	Binary Lifting
MST	Kruskal / Prim
Bipartite graph	BFS/DFS 2-coloring
DAG ordering	Topological Sort
Unweighted shortest path	BFS
0/1 weights	0-1 BFS
Non-negative weights	Dijkstra
Negative edges	Bellman-Ford
All-pairs shortest path	Floyd-Warshall
Directed SCCs	Kosaraju / Tarjan
2-SAT	SCC
Max flow	Dinic
Bipartite matching	Kuhn / Hopcroft-Karp
Euler path/circuit	Hierholzer
Pattern matching	KMP / Z
Longest palindrome	Manacher
Many patterns	Aho-Corasick
Substring automaton	Suffix Automaton
Minimum rotation	Booth
Prime preprocessing	Sieve / SPF
Huge 64-bit factorization	Miller-Rabin + Pollard-Rho
nCr mod prime	Factorials / Lucas
Linear recurrence	Matrix Expo / Fast Doubling
Subset over masks	SOS DP
Interval DP optimization	Knuth / D&C DP
Dynamic line minimum	Li Chao
Binary search on answer	First/Last True
Bounded subset sum	Bitset + binary splitting